

پروژه ۱

دستیار آموزشی: معین اثنی عشری

عنوان:

طراحی دوربین هوشمند

نوع پروژه:

عملی

در این پروژه با استفاده از ESP-CAM یک دوربین هوشمند جهت اعلام فرمان با حرکات دست و کنترل یک رله و Servo Motor طراحی خواهید کرد. روش کار به این صورت است که اگر کاربر دست خود را به صورت کاملاً باز به مدت ۵ ثانیه ثابت نگه دارد، باید با یک تأخیر چندثانیه‌ای تصویر دوبین ثبت و ذخیره شود. اگر کاربر دست خود را به صورت افقی حرکت دهد (از راست به چپ یا از چپ به راست)، با مشاهده و تشخیص این حرکت باید Servo Motor در همان جهت حرکت کند. همچنین در صورتی که کاربر دست خود را بین ۲ الی ۴ ثانیه باز نگه داشته و سپس دست خود را مشت کند، باید رله خاموش شود و در صورتی که کاربر دست خود را ابتدا به صورت مشت کرده بین ۲ الی ۴ ثانیه جلوی دوربین نگه داشته و سپس مشت خود را باز کند، رله باید روشن شود. جهت نمایش بهتر کار کردن رله، از یک چراغ LED استفاده کنید که با متصل شدن رله روشن شود. همچنین با استفاده از یک نمایشگر 7-Segment تعداد دستورات اجرا شده تا هر لحظه را نمایش دهید.

سیستم‌های نهفته عمدتاً در محیط‌هایی کار کنند که دسترسی به برق پایدار وجود ندارد. به همین دلیل این سیستم‌ها قابلیت‌های گوناگونی را ارائه داده‌اند تا بتوان با کمترین توان ممکن و با اتکا به منابعی مانند باتری، به مدت طولانی به کار خود ادامه دهند. در این پروژه قصد داریم یک سیستم کاربردی و کم‌توان برای جمع‌آوری و مانیتورینگ داده‌های محیطی ارائه کنیم.

صورت پروژه:

در این پروژه باید با استفاده از ESP32 و سنسور دما و رطوبت و یک LED، سیستمی طراحی کنید که در بازه زمانی دلخواه مثلاً هر یک ساعت یک بار، مقدار سنسور را خوانده و تاریخچه‌ای از اطلاعات ۲۰ دفعه خواندن قبلی سنسور همراه با زمان واقعی خواندن هر یک را با استفاده از ایمیلی دارای قالب HTML به لیستی از کاربرها ایمیل کند. همچنین هر ۱۰ ثانیه باید LED یک چشمک کوتاه بزند که نشان‌دهنده فعال بودن سیستم است. در بین هر دو بازه ارسال سیستم باید با قرارگیری در فاز کم‌توان، کمترین میزان انرژی مصرفی ممکن را داشته باشد. همچنین با فشردن یک دکمه در هر زمان سیستم باید وارد حالت تنظیمات شده، یک Access Point ایجاد کرده و منتظر اتصال یک دستگاه کاربر مدیر بماند. به منظور افزایش امنیت پس از اتصال، کاربر مدیر باید با کی‌پد در طرف ESP32 یک پین ۴ رقمی وارد کرده و همان پین را برای تأیید در صفحه وب نیز وارد کند بدین صورت از اتصال دستگاه‌های مزاحم در هنگام تغییر تنظیمات جلوگیری می‌شود. پس از تأیید پین، صفحه وب باید اطلاعات لحظه‌ای سنسور و تاریخچه آن را در صفحه وب نشان دهد. همچنین در این حالت کاربر مدیر باید بتواند اطلاعات کاربران مقصد را مشاهده کرده، عضوی را حذف کرده، تغییر داده یا اضافه کند. سپس با فشردن دوباره کلید سیستم باید دوباره وارد فاز کم‌توان شده و به جمع‌آوری اطلاعات بپردازد.

- قالب HTML ایمیل شما باید دارای فایل استایل CSS بوده و حداقل یک عکس در خود داشته باشد. داده عکس باید به صورت محلی همراه با فایل HTML ارائه شود و نباید یک لینک از اینترنت باشد. همچنین ایمیل باید نام کاربری مخاطب را هم در خود داشته باشد.
- به منظور نمایش اطلاعات لحظه‌ای باید از یک المان گرافیکی مانند یک gauge استفاده کنید.
- برنامه شما باید حداقل تا ۲۰ کاربر را پشتیبانی کرده و نام کاربری و ایمیل آنها را ذخیره داشته باشد.
- علاوه بر موارد فوق در تنظیمات سیستم، کاربر باید بتواند بین مد چشمک زن و مد پیوسته روشن ماندن LED انتخاب کند.

هدف این پروژه، ساخت یک سیستم هوشمند برای ارسال امن پیام‌های صوتی از یک نقطه به نقطه دیگر است. این سیستم از سه بخش اصلی تشکیل شده که با هم کار می‌کنند: دستگاه فرستنده، تعدادی Base Station به عنوان واسطه، و یک دستگاه گیرنده نهایی.

اجزای سیستم

بخش اول: لایه دستگاه فرستنده این بخش یک دستگاه بیسیم دستی است که با یک میکروکنترلر ESP32 ساخته شده است. وظیفه اصلی این لایه، تأیید هویت کاربر با رمز صوتی، ضبط پیام و آماده‌سازی آن برای ارسال امن است.

- میکروفون: صدای کاربر را برای رمز عبور و پیام ضبط می‌کند.
- پردازشگر ESP32: رمز عبور صوتی را به صورت محلی بررسی می‌کند و در صورت تأیید، پیام صوتی را به چند بخش تقسیم کرده و هر بخش را به صورت جداگانه رمزنگاری می‌کند.
- پوش باتن‌ها و LED ها: برای تعامل با کاربر (فعال‌سازی، ضبط، ارسال) و نمایش وضعیت دستگاه.

بخش دوم: لایه Base Stations، شبکه میانی سیستم است. این لایه شامل چندین دستگاه ساده است که فقط نقش پستچی یا روتر را دارند و پیام‌ها را به مقصد می‌رسانند. برای شبیه‌سازی آنها از دو یا سه ESP32 استفاده خواهیم کرد.

- ارتباط با فرستنده: بسته‌های رمزنگاری شده را از دستگاه فرستنده دریافت می‌کنند.
- مسیریابی ایستا: هر ایستگاه بدون خواندن محتوای بسته، آن را فوراً به سمت آدرس از پیش تعیین شده‌ی دستگاه گیرنده ارسال می‌کند. این ایستگاه‌ها هیچ اطلاعاتی را ذخیره یا پردازش نمی‌کنند.

بخش سوم: یک گوشی موبایل گیرنده این بخش و مقصد نهایی پیام خواهد بود که تمام قطعات پازل را کنار هم می‌چیند. شما از طریق این دستگاه می‌توانید:

- بخش‌های مختلف پیام را از Base Station های متفاوت دریافت کنید.
- بسته‌ها را رمزگشایی کرده و بر اساس اطلاعات همراهشان، آن‌ها را به ترتیب صحیح بچینید.
- پیام صوتی کامل و بازسازی شده را برای شنیدن آماده کنید.

پروژه ۴

دستیار آموزشی: امیرمحمد صالح

عنوان:

تصویرسازی سه بعدی

نوع پروژه:

عملی

در این پروژه، هدف ساخت یک سامانه‌ی تصویربرداری سه‌بعدی ساده با استفاده از پردازش تصویر و کنترل حرکتی دقیق است. در این طرح، یک پلتفرم چرخان روی یک سروو موتور نصب می‌شود که قابلیت چرخش ۳۶۰ درجه دارد. جسم مورد نظر روی این پلتفرم قرار می‌گیرد و یک دوربین ثابت در کنار آن مستقر است. هنگام چرخش تدریجی پلتفرم، در هر زاویه مشخص از جسم تصویر گرفته می‌شود و در نتیجه مجموعه‌ای از فریم‌ها از زوایای مختلف به‌دست می‌آید. این تصاویر به‌عنوان ورودی نرم‌افزار بازسازی سه‌بعدی (Photogrammetry) مورد استفاده قرار می‌گیرند تا مدل سه‌بعدی جسم بازسازی شود.

بازسازی سه‌بعدی روی خودِ Raspberry Pi به‌دلیل محدودیت سخت‌افزاری ممکن است کند باشد، اما می‌توان تصاویر را به سیستم قدرتمندتری منتقل و در آنجا پردازش کرد. خروجی نهایی باید یک تصویر سه بعدی از جسم تصویر برداری شده باشد که برای سیستم ما ارسال شده است.

پروژه ۵

دستیار آموزشی: احمد شمس

عنوان:

کنترل دمای اتاق سرور به صورت اتوماتیک

نوع پروژه:
عملی

برای کنترل دمای اتاق سرور از یک سنسور تشخیص دما و رطوبت استفاده می کنیم تا فن ها با سرعت مشخصی به حرکت درآیند، به این صورت که مقدار دمای محیط را با استفاده از این سنسور توسط یک برد ESP32 تشخیص داده می شود. سپس این مقدار توسط پروتکل های ارتباطی Master-Slave به صورت وایرلس برای یک چیپ Rasbery Pi ارسال می شود. بر اساس دمای تشخیص داده شده در محیط و قاعده تعیین شده در Rasbery Pi ، میزان حرکت فن ها را تعیین کرده و برای برد ESP32 ارسال می کند. برد ESP32 فن ها را به حرکت درآورده و دائماً فیدبک به Rasbery Pi ارسال می کند. در سمت Rasbery Pi اینطور فرض می شود که کاربری مسئول بررسی وضعیت سیستم می باشد. پس باید نمایشگری وجود داشته باشد تا اطلاعات کلی سیستم شامل سرعت فن ها و یا دمای اتاق سرور را گزارش دهد. همچنین در صورت تجاوز دما از حد مجاز باید خطاری صوتی-تصویری به کاربر بدهد که این اخطار شامل پیغام صفحه نمایش، روشن کردن چراغ هشدار قرمز و آژیر از بلندگو می باشد.

برای تعیین سرعت حرکت، در دمای ۱۶ درجه که دمای ایده آل محیط می باشد، فن ها خاموش در نظر گرفته می شوند، و به ازای هر ۲ درجه افزایش دما، ۵ درصد حداکثر سرعت موتور ها به سرعت حرکت آن ها اضافه می شود برای مثال در دمای ۲۴ درجه، سرعت حرکت موتور ها باید ۲۰ درصد سرعت کل و در دمای ۵۰ درجه ۸۵ درصد سرعت کل باید تنظیم شود. حد مجاز برای دما ۶۰ درجه سانتیگراد در نظر گرفته می شود.

یکی از مهمترین کاربردهای سیستم‌های نهفته در کنترل پارامترهای محیطی است. این پارامترها عمدتاً با یک سنسور از محیط مطالعه شده و با انجام یکسری محاسبات ریاضی بر روی آن‌ها تبدیل به سیگنال برای فعال کردن محرک‌های متصل می‌شوند. این محرک‌ها با اثرگذاری بر روی محیط سبب تغییر در پارامترهای محیطی می‌شوند. در مسأله کنترل هدف آن است که این تعامل با محیط به گونه‌ای انجام شود که یک پارامتر مشخص در محدوده‌ی خاصی تثبیت شود.

صورت پروژه:

در این پروژه قصد داریم که زاویه‌ی شفت یک موتور DC را کنترل کنیم. بدین منظور باید یک عدد سنسور gyroscope مانند MPU-6050 را به شفت موتور DC متصل کرده، و بدین صورت در هر لحظه زاویه شفت موتور را بررسی کنید. پس از آن باید با ارسال یک عدد زاویه از طریق کامپیوتر به برد کنترل کننده (Arduino) یک سیگنال کنترلی برای موتور تولید کنید به طوری که شفت موتور در زاویه مورد نظر قرار بگیرد. به منظور انجام این عملیات از باید از کنترل کننده‌های مرتبط با پارادایم زمان پیوسته و معادلات دیفرانسیل استفاده کنید.

برای انتخاب و تهیه موتور سراغ موتوهای گیربکس دار با گشتاور بالاتر و سرعت چرخش کمتر بروید.

***توجه: به منظور اتصال سنسور به شفت از آسیب زدن به سنسور یا استفاده از چسب‌ها و اتصالات دائمی خودداری کنید.**

این پروژه یک بازی تعاملی است که در آن، بازیکن با در دست داشتن یک امبدد دیوایس، از طریق دویدن و حرکت فیزیکی، کاراکتر خود را در یک اپلیکیشن موبایل یا کامپیوتر کنترل می‌کند. ویژگی کلیدی سیستم، حساسیت تطبیقی است که تأثیر حرکت فیزیکی را بر اساس اندازه محیط بازی تنظیم کرده و تجربه را پویا می‌سازد.

اجزای سیستم

بخش اول: امبدد دیوایس حرکات بازیکن را ثبت کرده و به بازی منتقل می‌کند.

- میکروکنترلر ESP32
- سنسور تشخیص دویدن MPU-6050 : برای تشخیص سرعت دویدن.
- کنترلر جهت Keypad : برای کنترل چرخش به چپ و راست.

بخش دوم: موبایل یا کامپیوتر که بازی را اجرا کرده و بازیکن با آن تعامل دارد.

- پلتفرم: یک اپلیکیشن موبایل یا یک برنامه ساده با HTML/CSS.
- منطق بازی: این قسمت تمام منطق بازی، از جمله هوش مصنوعی دشمن (یک RL ساده قابل قبول است) و ویژگی "حساسیت تطبیقی" را اجرا می‌کند.

نحوه کارکرد در عمل

- بازی روی اپلیکیشن موبایل یا در مرورگر وب باز می‌شود. کد جاوااسکریپت یا اپلیکیشن، ابتدا اندازه نقشه را تحلیل کرده و ضریب حساسیت را تنظیم می‌کند. بازیکن امبدد دیوایس که با میکروکنترلر ESP32 ساخته شده است را به خود متصل می‌کند.
- بازیکن با دویدن سرعت کاراکتر و با دکمه‌های کی‌پد جهت آن را کنترل می‌کند.
- دستگاه ESP32 داده‌های "سرعت" و "جهت" را از طریق Wi-Fi به موبایل/کامپیوتر ارسال می‌کند.
- برنامه بازی این داده‌ها را دریافت کرده و موقعیت کاراکتر بازیکن را روی صفحه به‌روزرسانی می‌کند.
- همزمان، برنامه بازی فاصله شما تا دشمن را محاسبه کرده و با نزدیک‌تر شدن دشمن، صدای پای آن را بلندتر پخش می‌کند تا هیجان و حس خطر افزایش یابد.

یک سامانه‌ی رایانش دو لایه‌ی کاربر و لبه داریم که در این سامانه هر کاربر یک دستگاه IoT است و قرار است وظایفی که به صورت استنتاج DNN هستند را روی شبکه تخصیص دهند. روی هر دستگاه IoT و سرور لبه یک مدل DNN وجود دارد که هر مدل از دقت (accuracy) مشخصی برخوردار است. به طور معمول مدل DNN دستگاه‌های IoT نسبت به مدل روی سرورها از دقت پایین‌تری برخوردار است. همچنین هر دستگاه IoT یک نرخ مصرف انرژی دارد که این نرخ ضربدر تعداد کلاک‌های وظیفه‌ای که به آن تخصیص داده می‌شود برابر با مصرف انرژی دستگاه IoT برای آن وظیفه می‌شود. همچنین برای تخصیص دادن وظایف روی سرورها به ازای هر لینکی که به یک سرور وجود دارد یک نرخ مصرف انرژی وجود دارد که این نرخ ضربدر اندازه‌ی داده‌ی وظیفه‌ای که ارسال می‌شود برابر با مصرف انرژی ارسال وظیفه به آن سرور می‌شود.

هر دستگاه IoT که در سامانه وجود دارد به همراه لینک‌هایی که دستگاه‌های IoT را به سرورهای لبه وصل می‌کنند با احتمال نا مشخصی ممکن است دچار خطایی شود که نتواند وظیفه‌ی استنتاج خود را به اتمام برساند. در نتیجه ما در این سامانه با چالش عدم قطعیت در رابطه با قابلیت اطمینان دستگاه‌های IoT و قابلیت اطمینان سرورهای لبه روبرو هستیم و باید بتوانیم با تخصیص دادن وظایف روی چند دستگاه و افزایش افزونگی قابلیت اطمینان وظایف خود را بالا ببریم. هدف این پروژه شبیه سازی سناریوی تعریف شده برای یک دستگاه IoT به همراه چندین سرور است. وظیفه‌ی دستگاه IoT تخصیص وظایف استنتاج DNN روی سرورها یا انجام این وظایف به صورت محلی (local) است؛ به گونه‌ای که پارامتر مصرف انرژی از Ebug که بودجه‌ی انرژی است کمتر شود و پارامترهای قابلیت اطمینان و دقت مدل DNN پیشینه شود.

از آن جایی که کمینه کردن انرژی و بیشینه کردن قابلیت اطمینان و دقت مدل DNN با همدیگر در تقابل هستند، باید الگوریتم لرنینگ قابل تنظیم برای دستگاه IoT طراحی شود که با در نظر گرفتن یک پارامتر قابل تنظیم (مانند V) بتواند اهمیت مصرف انرژی نسبت به قابلیت اطمینان و دقت مدل را بسنجد و بر اساس آن وظایف را تخصیص دهد.

مثال (۱) اگر قصد ما مصرف انرژی پایین باشد می‌توانیم تعداد تخصیص وظایف را پایین بیاوریم تا انرژی کمتری در سامانه مصرف شود؛ اما در مقابل آن قابلیت اطمینان و دقت مدل‌های DNN پایین می‌آید.

مثال (۲) اگر قصد ما افزایش قابلیت اطمینان و دقت مدل DNN باشد ما می‌توانیم تعداد تخصیص وظایف را بر روی سرورهایی که دقت بالاتری دارند افزایش بدهیم که با این کار ممکن است مصرف انرژی بیشتر از بودجه‌ی انرژی بشود.

نکته‌ی آخر: همانطور که گفته شد هر دستگاه IoT و سرورها با احتمال نامشخصی ممکن است حین پردازش وظایف دچار خطا شوند (خطای مدل DNN، خطای سخت افزاری، خطای شبکه، ...). به ازای هر سرور و دستگاه IoT یک لیست از **مشاهدات قبلی** هر دستگاه وجود دارد که نشان می‌دهد این دستگاه در وظایف قبلی که به آن‌ها تخصیص داده شده بود چگونه عمل کرده است. در این لیست عدد صفر به معنی ناموفق بودن وظیفه و عدد یک به معنی انجام شدن وظیفه است.

الگوریتم طراحی شده باید بتواند احتمال انجام شدن وظایف را با توجه به مشاهدات قبلی تخمین بزند و از این طریق جهت افزایش قابلیت اطمینان وظایف خود را روی دستگاه‌هایی تخصیص دهد که بیشترین احتمال انجام شدن وظایف را دارند.

در مرحله‌ی اول پروژه نیاز است تا محیط شبیه‌سازی پروژه پیاده‌سازی شود و می‌بایست موجودیت‌های پروژه (دستگاه IoT، سرورهای لبه، مدل‌های DNN و ...) طراحی شوند. پیاده‌سازی در این فاز باید به گونه‌ای باشد که محیط شبیه‌سازی بتواند یک وظیفه‌ی DNN را روی یک سرور لبه ارسال کند و نتیجه‌ی ارسال این وظیفه را نمایش دهد. در مرحله‌ی بعد نیاز است تا الگوریتم توزیع وظایف با هدف قابلیت اطمینان و دقت مدل‌های DNN و انرژی مصرفی پیاده‌سازی شود. همچنین یک الگوریتم دیگر می‌بایست به عنوان الگوریتم پایه طراحی شود تا با الگوریتم طراحی شده مقایسه شود. تهیه و ارسال گزارش کار و کد پیاده‌سازی برای هر مرحله از کار الزامی است.

پیاده‌سازی تمامی موارد گفته‌شده، تولید تمام خروجی‌های خواسته‌شده و تهیه گزارش پایانی برای این پروژه الزامی است. در گزارش پایانی می‌بایست کدهای پیاده‌سازی شده خط به خط توضیح داده شوند و نتایج بدست آمده در گزارش قرار داده شده و تحلیل شوند.

در سامانه های نهفته چند هسته ای، بسیاری از کاربردهای بی درنگ دارای ساختار موازی بوده و برای اجرا نیاز دارند که چند هسته به صورت همزمان در اختیار آن ها قرار گیرد. این نوع وظایف که با عنوان وظایف گنگ شناخته می شوند، از وظایف سنتی مستقل متمایز هستند، زیرا موفقیت آن ها فقط به اجرای صحیح یک کار روی یک هسته وابسته نیست، بلکه به اختصاص همزمان چند هسته، اجرای صحیح همه نخها، تبادل صحیح داده میان نخها، همگام سازی موفق، و رعایت deadline وابسته است. در این پروژه هدف آن است که یک مدل عمومی برای قابلیت اطمینان وظایف گنگ در سامانه های نهفته ارائه شود و با استفاده از شبیه سازی بررسی گردد که چگونه ساختار همکاری بین نخها، الگوی ارتباط، نقاط barrier، و خرابی منابع مشترک، بر قابلیت اطمینان کل وظیفه اثر می گذارند.

روش اجرا

در این پروژه دانشجو باید یک شبیه ساز در Python طراحی کند که بتواند مجموعه ای از وظایف گنگ را روی یک سامانه چند هسته ای اجرا و ارزیابی کند. برای روش زمان بندی میتوان از یک روش ساده مانند یکی از موارد زیر استفاده کرد:

Gang EDF - Gang Fixed Priority

سناریوهای مقایسه

برای آن که پروژه قابلیت تبدیل به مقاله داشته باشد، باید حداقل سه مدل با یکدیگر مقایسه شوند:

مدل ۱: مدل سنتی : فقط خرابی هسته ها را لحاظ می کند و ارتباط و همگام سازی را نادیده می گیرد.

مدل ۲: مدل توسعه یافته : خرابی هسته ها، ارتباطات، barrier و deadline را لحاظ می کند.

مدل ۳: مدل کامل : علاوه بر همه موارد فوق، خرابی های مشترک منابع اشتراکی را نیز در نظر می گیرد.

خروجی های مورد انتظار

- نمودار قابلیت اطمینان بر حسب اندازه گنگ m_i
- نمودار قابلیت اطمینان بر حسب تعداد barrier ها
- نمودار قابلیت اطمینان بر حسب شدت تبادل داده بین نخها
- نمودار قابلیت اطمینان بر حسب نرخ خرابی مشترک
- نمودار احتمال از دست رفتن deadline بر حسب بار سیستم
- نمودار مقایسه ای مدل کلاسیک و مدل پیشنهادی از نظر قابلیت اطمینان
- نمودار تحلیل حساسیت قابلیت اطمینان نسبت به پارامترهای مختلف سیستم

در این پروژه، هدف طراحی و پیاده‌سازی یک روش زمان‌بندی وظایف بی‌درنگ گنگ در ROS 2 است. در این روش، چند callback مرتبط با هم به‌صورت یک مجموعه در نظر گرفته می‌شوند و باید به‌طور هماهنگ و هم‌زمان روی چند هسته اجرا شوند. از آنجا که در ROS 2 اجرای callback های subscription ، timer ، service و action توسط executor انجام می‌شود، در این پروژه باید یک executor سفارشی طراحی شود یا سازوکار executor فعلی تغییر کند تا این مجموعه وظایف فقط زمانی اجرا شوند که همه هسته‌های موردنیاز آن‌ها آزاد باشند. برای هر وظیفه نیز پارامترهایی مانند زمان آزادسازی یا دوره تناوب، deadline، زمان اجرای بدترین حالت و تعداد هسته موردنیاز مشخص می‌شود. سپس یکی از سیاست‌های زمان‌بندی مانند Gang-EDF یا Gang-Fixed-Priority پیاده‌سازی و بررسی خواهد شد. همچنین برای ساده‌تر شدن طراحی و کاهش تداخل بین وظایف، می‌توان از روشی استفاده کرد که در هر لحظه فقط یک gang اجرا شود.

فعالیت‌های مورد انتظار:

- راه‌اندازی محیط ROS 2 روی Linux و اجرای آن روی یک سکوی نهفته چندهسته‌ای مانند Raspberry Pi 4 یا یک سیستم چندهسته‌ای مشابه.
- طراحی مجموعه‌ای از وظایف بی‌درنگ در قالب node و callback در ROS 2 .
- تعریف callback group های مناسب برای مشخص کردن callback هایی که باید به‌صورت gang اجرا شوند.
- پیاده‌سازی یک Custom Executor یا تغییر در سازوکار اجرای callback ها برای پشتیبانی از Gang Scheduling .
- پیاده‌سازی یکی از سیاست‌های زمان‌بندی مانند Gang-EDF یا Gang-Fixed-Priority .
- ثبت زمان آزادسازی، زمان شروع، زمان پایان، response time و تعداد deadline miss برای هر وظیفه.
- مقایسه عملکرد روش پیشنهادی با executor پیش‌فرض ROS 2 یا MultiThreadedExecutor .
- بررسی اثر تعداد هسته‌ها، میزان بار سیستم، و درجه موازی‌سازی وظایف بر عملکرد زمان‌بندی.

خروجی‌های مورد انتظار:

- پیاده‌سازی یک نمونه سامانه ROS 2 که در آن callback های مرتبط به‌صورت gang و بی‌درنگ اجرا شوند.
- ارائه نتایج آزمایش برای معیارهایی مانند deadline miss ratio ، response time ، jitter و CPU utilization .
- تحلیل این موضوع که آیا اجرای گنگ موجب بهبود پیش‌بینی‌پذیری زمانی و کاهش تداخل بین وظایف می‌شود یا خیر.
- ارائه گزارش کامل از مراحل طراحی، پیاده‌سازی، آزمایش و تحلیل نتایج.

تشخیص چهره با استفاده از یادگیری عمیق یکی از کاربردهایی است که همواره در حوزه اینترنت اشیا مورد توجه بوده است. در این نوع کاربرد، هدف برچسب‌گذاری نام فرد بر روی چهره‌ی دریافتی از او از دوربین به صورت بلادرنگ است. تشخیص چهره در بسیاری از کاربردها از جمله کنترل ورود و خروج به محیط‌های مختلف مورد استفاده قرار می‌گیرد.

تشخیص چهره از طریق به کار گرفتن یک نوع CNN با روش یادگیری supervised صورت می‌گیرد. در این روش معمولاً از یک وسیله‌ی اینترنت اشیا استفاده می‌شود در حالی که در روش توزیع شده، از چندین وسیله برای تشخیص چهره استفاده می‌شود که این کار مستلزم پارتیشن‌بندی CNN و همچنین برقراری ارتباط موثر بین وسایل و همکاری بین آنها می‌باشد.

هدف این پروژه پیاده‌سازی تشخیص چهره‌ی توزیع شده است. برای این کار نیاز است که یک شبکه CNN را با استفاده از یک دیتاست مناسب به صورت توزیع شده آموزش دهید. محاسبات مربوط به آموزش این مدل باید به صورت توزیع شده انجام شود و در نهایت نتیجه‌ی انجام محاسبات aggregate شود. در انتخاب ساختار مدل، دیتاست و روش شبیه‌سازی محدودیتی وجود ندارد اما دقت کنید که شبیه‌سازی فرایند آموزش توزیع شده باید نزدیک به واقعیت باشد. در نهایت باید یک گزارش درباره‌ی تمام مفروضات پروژه، روش انتخاب شده و نتایج بدست آمده و تحلیل آن‌ها (به همراه کدها) ارائه دهید.

در این پروژه قصد داریم تا یک سیستم نظارت هوشمند برای پایش کیفیت هوا، دما، رطوبت، نور و سایر متغیرها در یک محیط ایجاد کنیم. همانطور که می‌دانیم داده‌های مرتبط با این متغیرها با استفاده از حسگرهای محیطی در دستگاه‌های IoT انجام می‌شوند اما با توجه به شرایط، در این پروژه از داده‌های شبیه‌سازی شده استفاده می‌کنیم.

ابتدا تعدادی داده‌ی مبتنی بر واقعیت تولید کنید که مقادیر خوانده شده از حسگرهای دما، هوا، رطوبت، نور و یا هر متغیر مرتبط دیگری را شبیه‌سازی کند. همچنین می‌توانید از دیتاست‌های آماده مانند [این دیتاست \(لینک داخلی!\)](#) استفاده کنید. در انتخاب دیتاست محدودیتی وجود ندارد و کافی است داده‌ها مبتنی بر واقعیت باشند. سپس با استفاده از یک اسکریپت، این داده‌ها را در interval هایی بخوانید و با استفاده از پروتکل MQTT در یک فرمت استاندارد به همراه زمان، در یک یا چند topic ارسال کنید. پس از آن یک سرور راه‌اندازی کنید که topic هایی که در آن‌ها داده ارسال شده است را بخواند و مقادیر خوانده شده را در یک داشبورد تحت وب نمایش دهد. همچنین یک مکانیزم هشداردهی مانند ارسال ایمیل در سرور در نظر بگیرید که در صورت رخ دادن اتفاقی غیر منتظره، اطلاع‌رسانی لازم را به فرد یا افراد مرتبط انجام دهد. در داشبورد طراحی شده باید هر کدام از متغیرهای در نظر گرفته شده به صورت گرافیکی نمایش داده شوند و وضعیت هر کدام از آن‌ها نیز مشخص باشد (برای مثال قرمز برای هوای بسیار آلوده، زرد برای آلودگی متوسط و سبز برای هوای تمیز).

در این پروژه به زمان‌بندی وظایف DAG روی یک سیستم ناهمگن با هدف مدیریت و کاهش مصرف انرژی می‌پردازد.

در این پروژه باید گراف وظایف با تعداد و وابستگی‌های متفاوت تولید شود. سپس زمان‌بندی وظایف با استفاده از سه روش HEFT، HDLF و Critical Path First انجام شود. همچنین سطح ولتاژ و فرکانس مناسب برای هر هسته با در نظر گرفتن deadline انتخاب می‌شود. علاوه بر این‌ها، در قدم بعد از روش slack reclamation برای کاهش بیشتر مصرف انرژی نیز استفاده شود.

نمودارها و خروجی‌های پروژه:

- مقایسه میزان مصرف انرژی برای تعداد وظایف متفاوت در سه روش زمان‌بندی
 - نمودار میزان موفقیت در رعایت deadline برای تعداد وظایف مختلف
 - نمودار توان مصرفی هر هسته در طول یک شبیه‌سازی
 - میزان utilization هسته‌ها برای مجموعه وظایف متفاوت
 - میانگین latency برای تعداد وظایف متفاوت
 - میزان انرژی صرفه‌جویی شده توسط روش slack reclamation نسبت به حالت بدون آن
- تمامی فرضیات در نظر گرفته شده، روند پیاده‌سازی، نتایج و تحلیل آن‌ها را در قالب یک گزارش به همراه کدها ارسال کنید.

یک پیاده‌رو یا معبر در یک نقطه‌ی شلوغ از شهر را نظر بگیرید که در آن یک دوربین نصب شده است. این دوربین در هر لحظه تصویری از پیاده‌رو را به یک مرکز مخابره می‌کند. هدف از وجود این سیستم این است که با توجه به تصویری که ارسال می‌شود، سطح شلوغی آن پیاده‌رو به صورت بی‌درنگ تخمین زده شود.

در این پروژه باید با استفاده از روش‌های یادگیری ماشین، یک مدل آموزش دهید که با دریافت تصویر یک پیاده‌رو، تعداد افراد موجود در تصویر را تخمین بزند و همچنین یک نقشه حرارتی از میزان تراکم افراد در نقاط مختلف پیاده‌رو را نمایش دهد. برای سادگی می‌توانید از مدل‌های آماده‌ی تشخیص اشیا مانند yolo و یا مدل‌های مشابه استفاده کنید و متناسب با نیاز خود آن‌ها را fine-tune کنید. دقت کنید که اجازه‌ی استفاده از مدل‌هایی که به طور اختصاصی برای این مسئله آموزش دیده شده‌اند را ندارید. همچنین در نظر داشته باشید که ممکن است در تصویر ارسال شده از دوربین، اشیا نامرتبط مانند ماشین‌ها و موتورهای نیز وجود داشته باشند. بنابراین روش شما باید با موفقیت تنها افراد را در نظر گرفته و تعداد آن‌ها را تخمین بزند. تمامی مفروضات پروژه، روند پیاده‌سازی و نتایج را در یک گزارش به همراه کدهای خود ارسال کنید.

یک در سامانه‌های نهفته و اینترنت اشیاء صنعتی، پیش‌بینی عمر مفید باقی‌مانده تجهیزات هوافضا، به‌ویژه موتورهای توربوفن، برای نگهداشت پیش‌بینانه و جلوگیری از خرابی‌های پرهزینه ضروری است. مجموعه‌داده C-MAPSS یک استاندارد معتبر برای تخمین عمر باقی‌مانده است که از داده‌های سری زمانی چندمتغیره موتورهای هم‌خانواده استخراج شده‌اند. با این حال، مدل‌های موجود همچون LSTM و Transformer با چالش‌هایی نظیر هزینه محاسباتی بالا، وابستگی زیاد به داده و حساسیت به خطاهای حسگر در شرایط عملیاتی مواجه هستند.

روش پیشنهادی

در این پروژه، مدل جدیدی معرفی شده است که این مدل از شبکه‌های کلموگروف-آرنولد برای شبیه‌سازی زوال موتور استفاده کرده و آن را با سه مؤلفه اصلی در نظر می‌گیرد:

۱. قید فیزیکی شاخص سلامت برای اطمینان از تفسیرپذیری مدل .
 ۲. تزریق اشکال حسگر و داده برای افزایش قابلیت اطمینان پذیری مدل در برابر خطاهای عملیاتی .
 ۳. تبدیل توابع اسپلاین به جدول جست‌وجو برای اجرای بی‌درنگ روی سخت‌افزارهای لبه .
- این رویکرد علاوه بر عملکرد علمی بهتر، از لحاظ پیاده‌سازی برای دستگاه‌های با منابع محدود (نهفته) نیز مناسب‌تر است.

خروجی‌های پروژه

- تحلیل دقت مدل: مقایسه عملکرد مدل پیشنهادی با مدل‌های پایه‌ای همچون LSTM و TCN از نظر معیارهای RMSE و Score
- تجزیه و تحلیل اثر شرایط عملیاتی و حالت خرابی: ارزیابی تاثیر نرمال‌سازی شرط-آگاه در مجموعه‌های چندشرطی و تحلیل تفاوت مدل در تشخیص حالت‌های خرابی مختلف (HPC و فن).
- مقاومت در برابر اشکال حسگر: ارزیابی مدل تحت شرایط مختلف نویز، رانش، چسبیدگی حسگر و ... و تحلیل اثر این اشکال بر دقت پیش‌بینی .
- تحلیل سازگاری فیزیکی: بررسی تغییرات شاخص سلامت و عمر مفید باقی‌مانده با استفاده از قید فیزیکی و جریمه یکنوایی .
- عدم قطعیت و تصمیم‌گیری محافظه‌کارانه: تخمین واریانس پیش‌بینی و ارزیابی پوشش بازه اطمینان ۹۰ درصد برای موتورهای آزمون .
- تحلیل زمان و حافظه: ارزیابی تاثیر طول پنجره‌های لغزان مختلف بر دقت مدل و بودجه حافظه مورد نیاز .
- TinyML و پیاده‌سازی بی‌درنگ: تبدیل مدل به LUT برای کاهش مصرف حافظه و بهبود زمان استنتاج.

پیاده‌سازی تمامی موارد گفته‌شده، تولید تمام خروجی‌های خواسته‌شده و تهیه گزارش پایانی برای این پروژه الزامی است. در گزارش پایانی می‌بایست کدهای پیاده‌سازی شده خط به خط توضیح داده شوند و نتایج بدست آمده بصورت نمودار در گزارش قرار داده شده و تحلیل شوند.

سند کامل پروژه

بعد از انتخاب این پروژه، یک فایل سند کامل از پروژه به شما ارائه خواهد شد که جزئیات روش پیشنهادی و مسئله به‌طور کامل در آن آورده شده است. همچنین مجموعه‌داده این پروژه در اختیار شما قرار خواهد گرفت.

باتری‌های لیتیوم-یون یکی از مهم‌ترین اجزای سامانه‌های ذخیره‌سازی انرژی، خودروهای الکتریکی، سامانه‌های حمل‌ونقل هوشمند و تجهیزات قابل حمل هستند و عملکرد، ایمنی و طول عمر آن‌ها به‌طور مستقیم به وضعیت حرارتی وابسته است. افزایش دمای باتری می‌تواند موجب تسریع فرایند پیرشدگی، کاهش ظرفیت قابل استفاده، افزایش مقاومت داخلی، افت بازده و در شرایط بحرانی، بروز ناپایداری‌های حرارتی شود. در مقابل، دماهای پایین نیز باعث کاهش توان قابل تحویل، افت راندمان و تغییر محسوس رفتار دینامیکی سلول می‌شوند. از این رو، پایش دقیق دمای باتری یکی از اجزای کلیدی در طراحی سامانه مدیریت باتری به‌شمار می‌رود. با این حال، در کاربردهای واقعی معمولاً تنها تعداد محدودی حسگر روی بدنه سلول یا درون محفظه نصب می‌شود و اندازه‌گیری همه کمیت‌های حرارتی مورد نیاز به‌صورت مستقیم امکان‌پذیر نیست. به همین دلیل، استفاده از حسگرهای مجازی مبتنی بر داده که بتوانند از روی سیگنال‌های در دسترس، رفتار حرارتی باتری را با دقت مناسب بازسازی کنند، از اهمیت بالایی برخوردار است. در این پروژه، هدف طراحی یک چارچوب داده‌محور و در عین حال آگاه از فیزیک برای تخمین، پیش‌بینی و پایش حرارتی باتری است. تمرکز اصلی پروژه بر دمای بدنه یا دمای کیس باتری قرار دارد، زیرا این کمیت در مجموعه داده انتخاب شده به‌صورت مستقیم اندازه‌گیری شده و بنابراین می‌توان مدل را بدون اتکا به فرض‌های غیرقابل اعتبارسنجی آموزش و ارزیابی کرد.

روش پیشنهادی

در این پروژه، یک مدل چندوظیفه‌ای مبتنی بر شبکه‌های کانولوشنی زمانی طراحی می‌شود که به‌صورت هم‌زمان چند وظیفه مکمل را انجام می‌دهد: تخمین دمای بدنه باتری در زمان جاری، پیش‌بینی دمای بدنه در یک افق کوتاه‌مدت، تشخیص اشکال حسگری و ناهنجاری‌های حرارتی و برآورد عدم قطعیت پیش‌بینی. انتخاب معماری شبکه‌های کانولوشنی زمانی به این دلیل است که این مدل‌ها برای داده‌های ترتیبی و سری‌های زمانی، ضمن حفظ وابستگی‌های زمانی، از پیچیدگی محاسباتی مناسب و قابلیت آموزش پایدار برخوردارند. ورودی مدل شامل سیگنال‌های ولتاژ، جریان، توان، آمپر ساعت، وات ساعت، دمای چمبر و نیز ویژگی‌های استخراج شده‌ای همانند نرخ تغییرات جریان، ولتاژ و اختلاف دمای باتری و محیط خواهد بود. برای افزایش قابلیت تعمیم و جلوگیری از تولید خروجی‌های غیرواقعی، یک قید فیزیکی ساده اما مؤثر نیز به تابع هزینه افزوده می‌شود تا روند زمانی دما با یک مدل حرارتی متمرکز سازگار باقی بماند. به بیان دیگر، مدل علاوه بر یادگیری از داده، به‌گونه‌ای آموزش می‌بیند که رفتار حرارتی آن با اصول پایه انتقال حرارت نیز همخوانی داشته باشد. برای آموزش و ارزیابی این چارچوب، از مجموعه داده Panasonic 18650PF استفاده می‌شود که یک مرجع مناسب برای این مسئله است و شامل سیگنال‌های الکتریکی و حرارتی، آزمون‌های HPPC، آزمون C/20، آزمون EIS و نیز پروفایل‌های رانندگی دینامیکی در دماهای محیطی ثابت و متغیر است. علاوه بر این، به‌منظور بررسی استحکام مدل در برابر شرایط غیرایده‌آل، انواع اشکال مصنوعی نظیر نویز، انحراف، رانش، چسبیدگی حسگر، داده گمشده و ناهنجاری‌های حرارتی به داده‌ها تزریق می‌شوند تا بخش تشخیص اشکال نیز به‌صورت کنترل شده آموزش ببیند. در نهایت، برای کاهش حجم مدل و حفظ قابلیت استفاده در سامانه‌های با منابع محدود، چندی‌سازی و آموزش آگاه از چندی‌سازی نیز در فرایند توسعه مدل لحاظ می‌شود.

خروجی‌های مورد نیاز

در پایان پروژه انتظار می‌رود یک سامانه کامل، قابل اجرا و مستند برای پایش حرارتی باتری ارائه شود که بتواند دمای بدنه باتری را با دقت مناسب تخمین بزند، تغییرات کوتاه‌مدت آینده را پیش‌بینی کند، خطاهای حسگری و ناهنجاری‌های حرارتی را شناسایی نماید و برای پیش‌بینی‌های خود میزان عدم قطعیت ارائه دهد. پیاده‌سازی کامل مراحل پیش‌پردازش داده، استخراج ویژگی‌ها، طراحی و آموزش مدل، اعمال قید فیزیکی، تزریق اشکال، ارزیابی نسخه چندی‌سازی شده و تحلیل نتایج الزامی است. گزارش نهایی باید شامل شرح دقیق فرایند آماده‌سازی داده، معماری مدل، تابع هزینه، تنظیمات آموزش، نوع اشکال تزریق شده، معیارهای ارزیابی و مقایسه نسخه‌های مختلف مدل باشد. همچنین لازم است نتایج به‌صورت نمودارهای تحلیلی ارائه شوند؛ از جمله نمودار دمای واقعی و تخمینی، نمودار پیش‌بینی آینده، نمودار عدم قطعیت، نمودار نمونه‌های خطا، و نمودارهای مقایسه‌ای برای بررسی اثر قید فیزیکی و چندی‌سازی. علاوه بر این، کدها باید به صورت کامنت شده و پیمانه‌ای باشد و دانشجویان باید در گزارش خود کدهای اصلی پیاده‌سازی را به‌صورت مستدل توضیح دهند و نشان دهند که هر بخش از مدل چه نقشی در عملکرد نهایی سامانه داشته است.

سند کامل پروژه

بعد از انتخاب این پروژه، یک فایل سند کامل از پروژه به شما ارائه خواهد شد که جزئیات روش پیشنهادی و مسئله به‌طور کامل در آن آورده شده است. همچنین مجموعه داده این پروژه در اختیار شما قرار خواهد گرفت

در خودروهای خودران، لایه‌های ادراک، تخمین حالت، تصمیم‌گیری و کنترل به داده‌های چندحسگری وابسته‌اند و هر اختلال در این زنجیره می‌تواند به افت ایمنی، کاهش قابلیت اطمینان و تصمیم‌گیری نادرست منجر شود. از این رو، مسئله تنها تشخیص وجود خرابی نیست، بلکه باید به‌صورت هم‌زمان مشخص شود کدام حسگر معیوب شده، نوع خرابی چیست و روند سلامت شبکه حسگری در افق آینده چگونه تغییر می‌کند. در این پروژه، یک چارچوب داده‌محور و مبتنی بر یادگیری عمیق برای تحلیل سه سیگنال گذرگاه خودرو شامل پدال گاز، زاویه فرمان و فشار ترمز بر پایه داده‌های واقعی A2D2 طراحی می‌شود. مسئله مورد نظر ماهیتی چندحسگری، چنداشکالی و وابسته به زمینه دارد. خرابی‌ها می‌توانند به‌صورت هم‌زمان در چند کانال ظاهر شوند و الگوهای متفاوتی همانند drift، hard-over، spike و erratic داشته باشند. علاوه بر این، رفتار یک سیگنال ممکن است در یک سناریوی رانندگی غیرعادی و در سناریوی دیگر طبیعی باشد؛ در نتیجه، مدل باید نسبت به زمینه رانندگی نیز حساس باشد. از سوی دیگر، کمیابی داده‌های واقعی run-to-failure، عدم توازن شدید میان داده‌های سالم و معیوب و دشواری برچسب‌گذاری مسیر تخریب، طراحی یک سامانه دقیق و تعمیم‌پذیر را به مسئله‌ای چالش‌برانگیز تبدیل می‌کند.

روش پیشنهادی

در این پروژه، یک چارچوب چندمرحله‌ای و سلسله‌مراتبی برای تشخیص، ایزوله‌سازی، شناسایی و پیش‌بینی سلامت حسگرها طراحی می‌شود. در گام نخست، داده‌های سه حسگر اصلی شامل پدال گاز، زاویه فرمان و فشار ترمز از مجموعه داده A2D2 استخراج شده و با نرخ نمونه‌برداری مشخص در پنجره‌های زمانی ثابت، برای تحلیل آماده‌سازی می‌شوند. سپس به‌منظور شبیه‌سازی سناریوهای خرابی در شرایط کنترل شده، چهار نوع خرابی متداول شامل drift، hard-over، erratic و spike و... به سیگنال‌های سالم تزریق می‌شود تا بستری استاندارد برای آموزش و ارزیابی مدل فراهم گردد. در مرحله دوم، برای آشکارسازی سریع وقوع خرابی، از یک شبکه کانولوشنی یک‌بعدی فشرده استفاده می‌شود که مستقیماً سیگنال خام چندکاناله را دریافت می‌کند. خروجی این بخش یک تصمیم دودویی درباره سالم یا معیوب بودن پنجره ورودی است. علاوه بر این، خروجی خطی لایه نهایی شبکه حفظ می‌شود تا در مراحل بعدی برای ساخت شاخص سلامت مورد استفاده قرار گیرد. در مرحله سوم، پس از آن که وجود خرابی در یک پنجره تشخیص داده شد، برای هر حسگر به‌صورت جداگانه مجموعه‌ای از ویژگی‌های حوزه زمان استخراج می‌شود. سپس این بردارهای ویژگی به یک شبکه عصبی عمیق چندکلاسه داده می‌شوند تا برای هر حسگر مشخص شود آیا سیگنال آن سالم است یا خیر و در صورت خرابی، نوع اشکال آن از میان کلاس‌های تعریف‌شده تعیین شود. در مرحله چهارم، به‌منظور ارزیابی وضعیت سلامت سامانه در طول زمان، از خروجی‌های خطی آشکارساز یک شاخص سلامت استخراج می‌شود. در مرحله نهایی، برای پیش‌بینی آینده شاخص سلامت و برآورد رفتار سامانه در افق‌های زمانی آتی، از مدل Temporal Fusion Transformer (TFT) استفاده می‌شود. خروجی این مدل در قالب چندک‌های P10، P50 و P90 ارائه می‌شود تا علاوه بر پیش‌بینی مقدار مرکزی، میزان عدم قطعیت نیز در ارزیابی سلامت آینده سامانه لحاظ شود.

خروجی‌های مورد نیاز

در این پروژه، انتظار می‌رود دانشجویان خروجی‌ها را به‌صورت علمی، تحلیلی و مستند ارائه کنند. این خروجی‌ها شامل گزارش معیارهای استاندارد بخش آشکارسازی نظیر Accuracy، Precision، Recall، F1-Score و AUC به همراه ماتریس درهم‌ریختگی است. همچنین لازم است دقت ایزوله‌سازی برای هر یک از سه حسگرها و نیز دقت شناسایی برای انواع مختلف خرابی به‌صورت مجزا گزارش شود. در بخش پایش سلامت، نمایش روند خروجی‌های خطی آشکارساز، سیگنال هموار شده و شاخص سلامت HI برای سناریوهای سالم و معیوب ضروری است. در بخش پیش‌بینی نیز باید خروجی‌های چندک‌های P10، P50 و P90 به همراه معیار NQL ارائه و تحلیل شوند. افزون بر این، مقایسه عملکرد روش پیشنهادی با روش‌های مینا، بررسی تعمیم‌پذیری مدل در سناریوی open-set مبتنی بر تفکیک جغرافیایی داده‌ها، و ارائه نمودارها، جداول و تحلیل علمی دقیق نتایج، از اجزای اصلی گزارش نهایی پروژه خواهد بود.

پیاده‌سازی تمامی موارد گفته‌شده، تولید تمام خروجی‌های خواسته‌شده و تهیه گزارش پایانی برای این پروژه الزامی است. در گزارش پایانی می‌بایست کدهای پیاده‌سازی شده خط به خط توضیح داده شوند و نتایج بدست آمده بصورت نمودار در گزارش قرار داده شده و تحلیل شوند.

سند کامل پروژه

بعد از انتخاب این پروژه، یک فایل سند کامل از پروژه به شما ارائه خواهد شد که جزئیات روش پیشنهادی و مسئله به‌طور کامل در آن آورده شده است. همچنین مجموعه داده این پروژه در اختیار شما قرار خواهد گرفت.

با گسترش سریع سامانه‌های اینترنت اشیاء در حوزه‌هایی همانند خانه هوشمند، سلامت، حمل‌ونقل و زیرساخت‌های حیاتی، امنیت ثابت‌افزار به یکی از مسائل اساسی در سامانه‌های نهفته تبدیل شده است. ثابت‌افزار به دلیل قرارگرفتن در نزدیک‌ترین لایه به سخت‌افزار، نقش تعیین‌کننده‌ای در راه‌اندازی دستگاه، مدیریت منابع، کنترل واسط‌ها و اجرای منطق پایه دارد؛ ازاین‌رو، آلودگی در این سطح می‌تواند پایداری بالایی داشته باشد، از ابزارهای متداول امنیتی پنهان بماند و دسترسی عمیقی به اجزای حیاتی دستگاه فراهم کند. مسئله اصلی آن است که بسیاری از روش‌های رایج تشخیص بدافزار برای فایل‌های سطح سیستم‌عامل یا نرم‌افزارهای متداول طراحی شده‌اند و در مواجهه با ثابت‌افزارهای نهفته، به‌ویژه در شرایطی که کد منبع در دسترس نیست، معماری‌ها متنوع‌اند و نمونه‌ها با فشرده‌سازی، بسته‌بندی یا مبهم‌سازی محافظت شده‌اند، کارایی کافی ندارند. علاوه بر این، روش‌هایی Control Flow Flattening، درج کد مرده، جایگزینی دستورها و استفاده از گزاره‌های مبهم، تحلیل ایستا و تشخیص مبتنی بر امضا را با دشواری جدی مواجه می‌کنند. بنابراین، چالش اصلی پروژه، طراحی یک چارچوب تحلیلی مقاوم، تعمیم‌پذیر و داده‌محور برای تشخیص بدافزارهای سطح ثابت‌افزار و کاهش اثر مبهم‌سازی در سامانه‌های نهفته است.

روش پیشنهادی

در این پروژه، یک چارچوب ترکیبی برای تحلیل و تشخیص بدافزار ثابت‌افزاری پیشنهاد می‌شود که بر تحلیل ایستا، یادگیری گراف و یادگیری عمیق چندنمایی استوار است. در گام نخست، تصویر خام ثابت‌افزار باز شده و بخش‌های اجرایی آن استخراج می‌شوند؛ سپس با استفاده از ابزارهای تحلیل دودویی، دستورها جداسازی شده و گراف جریان کنترل هر بخش ساخته می‌شود. به‌منظور کاهش وابستگی به معماری‌های خاص، از نمایش‌های انتزاعی شامل ساختار گراف جریان کنترل، توالی‌های opcode و تصویر خاکستری بایت‌های دودویی استفاده می‌شود. در ادامه، ویژگی‌های ساختاری گراف با استفاده از شبکه‌های کانولوشنی گرافی استخراج می‌شوند تا الگوهای رفتاری پایدار در برابر تغییرات نحوی شناسایی شوند. به‌صورت هم‌زمان، نمایش تصویری دودویی به یک شبکه کانولوشنی داده می‌شود تا الگوهای بایت‌سطحی، نواحی با آنتروپی غیرعادی و نشانه‌های فشرده‌سازی یا بسته‌بندی آشکار شوند. در مرحله بعد، بردارهای حاصل از شاخه‌گرافی و شاخه‌تصویری با یکدیگر ادغام شده و به طبقه‌بند نهایی برای تشخیص سالم یا آلوده بودن نمونه، و در صورت امکان شناسایی خانواده بدافزاری، ارسال می‌شوند. افزون بر این، یک مؤلفه ابهام‌زدایی مبتنی بر یادگیری برای بازسازی تقریبی ساختار تمیز گراف جریان کنترل در نظر گرفته می‌شود تا اثر روش‌های مبهم‌سازی کاهش یابد و تحلیل‌پذیری نمونه افزایش پیدا کند.

خروجی مورد انتظار

خروجی مورد انتظار این پروژه، پیاده‌سازی یک سامانه تحلیلی کامل برای تشخیص بدافزارهای سطح ثابت‌افزار و ارزیابی علمی عملکرد آن است. انتظار می‌رود دانشجو پس از استخراج داده‌ها و ساخت نمایش‌های مناسب، عملکرد مدل را با معیارهایی همانند Precision، Accuracy، Recall و F1-Score گزارش کرده و نتایج را در قالب جداول، نمودارها و ماتریس درهم‌ریختگی ارائه دهد. همچنین لازم است کیفیت ابهام‌زدایی ساختار گراف با معیارهایی همانند Graph Edit Distance بررسی شود و نمونه‌هایی از گراف جریان کنترل قبل و بعد از بازسازی تحلیل شوند. از سوی دیگر، ارزیابی مقاومت مدل در برابر انواع مبهم‌سازی، سنجش تعمیم‌پذیری آن در معماری‌های مختلف، و مقایسه با روش‌های مینا از جمله بخش‌های ضروری پروژه است. در نهایت، خروجی نهایی باید علاوه بر پیش‌بینی کلاس هر نمونه، شامل تحلیل علمی نقاط قوت و محدودیت‌های روش، بررسی زمان تحلیل، و ارائه تفسیری روشن از نقش هر مؤلفه در تصمیم نهایی سامانه باشد.

پیاده‌سازی تمامی موارد گفته‌شده، تولید تمام خروجی‌های خواسته‌شده و تهیه گزارش پایانی برای این پروژه الزامی است. در گزارش پایانی می‌بایست کدهای پیاده‌سازی شده خط به خط توضیح داده شوند و نتایج بدست آمده بصورت نمودار در گزارش قرار داده شده و تحلیل شوند.

سند کامل پروژه

بعد از انتخاب این پروژه، یک فایل سند کامل از پروژه به شما ارائه خواهد شد که جزئیات روش پیشنهادی و مسئله به‌طور کامل در آن آورده شده است.

در طراحی سیستم های نهفته و با کارایی بالا، درک دقیق رفتار زیرسیستم حافظه ضرورتی فنی برای بهینه سازی کد محسوب می شود. هدف این پروژه، سنجش مستقیم پارامترهای عملیاتی حافظه پنهان (Cache) در سطوح L1، L2 و L3 است. دانشجویان در این مسیر موظفند با تلفیق زبان C/C++ و دستورات اسمبلی سطح پایین، ابزاری دقیق جهت استخراج تأخیر (Latency) و ظرفیت کش در پردازنده های مدرن Intel یا AMD طراحی کنند.

چالش اصلی در اندازه گیری تأخیر در مقیاس نانو ثانیه، حذف اثرات جانبی معماری پردازنده و مدیریت نویز سیستم عامل است. در فاز نخست، جهت دستیابی به دقت در مقیاس سایکل، باید از شمارنده زمان سنج سخت افزاری (TSC) استفاده شود. در محیط های لینوکس، فراخوانی دستور اسمبلی rdtsc و در محیط ویندوز با کامپایلر MSVC، استفاده از توابع ذاتی (Intrinsics) مانند __rdtsc الزامی است. برای تضمین دقت، لازم است از موانع حافظه (Memory Barriers) همچون cpuid یا lfence پیش و پس از زمان سنجی استفاده شود تا از اجرای خارج از نوبت (Out-of-Order Execution) توسط خطلوله پردازنده جلوگیری شده و فرآیند سریال سازی (Serialization) به درستی انجام پذیرد.

در گام بعد، برای غلبه بر بهینه سازی های سخت افزاری نظیر Hardware Prefetcher که الگوهای دسترسی ترتیبی را پیش بینی می کنند، تکنیک Pointer Chasing باید پیاده سازی شود. در این روش، یک آرایه با ساختار لیست پیوندی درهم ریخته ایجاد می گردد که در آن هر گره حاوی آدرس گره بعدی در مکانی غیر مجاور است. جهت حذف کامل اثر پیش بینی گر، فواصل پرش در حافظه باید حداقل ۶۴ بایت) مطابق با سایز متعارف (Cache Line) در نظر گرفته شود. همچنین، برای به حداقل رساندن نویز ناشی از Context Switches و مهاجرت وظیفه بین هسته ها، دانشجو باید ترد اجرایی را با استفاده از توابعی نظیر SetThreadAffinityMask در ویندوز یا sched_setaffinity در لینوکس، به یک هسته فیزیکی خاص محدود (Pin) نماید.

پیاده سازی نهایی باید دسترسی به حافظه را در بازه های متغیر) از ۱ کیلوبایت تا فراتر از ظرفیت (L3 آزمایش کرده و با تکرار هزاران باره هر آزمایش، میانگین آماری دقیقی استخراج کند. خروجی نهایی پروژه، نموداری است که در آن محور افقی نشان دهنده ابعاد حافظه مورد آزمایش (در مقیاس لگاریتمی) و محور عمودی بیانگر زمان دسترسی بر حسب سایکل پردازنده است. انتظار می رود در این نمودار، رفتار پله ای (Step-like behavior) در نقاط مرزی سطوح مختلف کش مشاهده شود. تطابق این پله های تجربی با مشخصات فنی استخراج شده از ابزارهایی نظیر CPU-Z، معیار اصلی صحت سنجی پروژه خواهد بود.

دانشجو موظف است کدهای منبع را با رعایت اصول مستند سازی فنی و ذکر دقیق بخش های اسمبلی ارائه دهد. گزارش نهایی باید شامل تحلیل عمیق نمودار استخراج شده، تشریح چگونگی غلبه بر بهینه سازی های کامپایلر و تبیین تفاوت های مشاهده شده میان نتایج عملی و مستندات سخت افزاری باشد. هرگونه انحراف در نمودار پله ای باید از منظر معماری کامپیوتر و تأثیر نویز سیستم عامل تحلیل گردد.

بهینه‌سازی مصرف توان در سیستم‌های نهفته، نیازمند عبور از روش‌های ایستا و اکتشافی (Heuristics) به سمت راهکارهای انطباق‌پذیر است. این پروژه یک واحد مدیریت توان خودکار مبتنی بر Reinforcement Learning (RL) و الگوریتم Q-learning را برای پیاده‌سازی DVFS در یک محیط تک‌هسته‌ای (Single-core) طراحی می‌کند تا تعادلی میان بهره‌وری انرژی و رعایت محدودیت‌های عملکردی ایجاد کند.

شما باید یک فریم‌ورک شبیه‌سازی رویداد-محور (Event-driven) برای پردازنده‌ای با سطوح فرکانسی (۰.۲۵، ۰.۵، ۰.۷۵ و ۱.۰) توسعه دهید. مصرف توان طبق رابطه $P \propto V^2 f$ مدل‌سازی شده و ولتاژ به صورت خطی با فرکانس تغییر می‌کند. هسته مدیریت توان، یک الگوریتم Q-learning جدولی (Tabular) است که با تعریف فضای حالت (State space) مناسب برای ثبت پویایی‌های سیستم و فشار زمان‌بندی (Scheduling pressure) و استفاده از استراتژی ϵ -greedy، فرکانس بهینه را انتخاب می‌کند. تابع پاداش باید شکست‌های Real-time (تجاوز از ددلاین) را با جریمه‌های سنگین و رعایت ددلاین‌ها را با پاداش منفی انرژی مصرفی (برای بهینه‌سازی توان) مدل‌سازی کند. مقادیر Q با نرخ یادگیری (α) و فاکتور تخفیف (γ) به‌روز می‌شوند.

ارزیابی عامل RL باید در برابر بارهای کاری متنوع و با استفاده از الگوریتم UUniFast برای تولید Taskset‌های تناوبی با ددلاین‌های ضمنی (Deadline = Period) انجام شود. دوره‌های Task باید از مجموعه $\{100, 200, 500, 1000\}$ میلی‌ثانیه انتخاب گردند. فریم‌ورک را در تمامی ترکیب‌های Utilization (۰.۳، ۰.۵، ۰.۷، ۰.۹) و تعداد Task (۵، ۱۰، ۱۵، ۲۰) ارزیابی کنید. به دلیل ماهیت تصادفی (Stochastic) یادگیری، برای هر ترکیب حداقل ۱۰۰ Taskset منحصربه‌فرد شبیه‌سازی و میانگین‌گیری شود تا اعتبار آماری نتایج حفظ گردد. نهایتاً، عملکرد رویکرد Q-learning را با سیستم‌های Baseline (فرکانس حداکثر) و DVFS اکتشافی مقایسه کنید.

نتایج شبیه‌سازی باید در قالب ۵ نمودار زیر تحلیل و ارائه شود:

۱. نمودار پاداش تجمعی (Cumulative reward) در اپیزودهای آموزشی متوالی که روند انتقال عامل از وضعیت شکست در ددلاین‌ها به سمت همگرایی روی سیاست بهینه و حداقل‌سازی مصرف انرژی را نشان می‌دهد.

۲. نمودار مصرف انرژی نرمال‌سازی شده برای روش‌های Baseline، Heuristic و RL در تمامی Utilization‌های هدف (۰.۳، ۰.۵، ۰.۷، ۰.۹) جهت بررسی بنچمارک ۲۰٪ صرفه‌جویی توان.

۳. نمودار درصد خطای ددلاین (Deadline miss percentage) برای تمامی روش‌ها در سطوح مختلف Utilization و تعداد Task جهت تأیید رعایت محدودیت‌های زمانی سیستم.

۴. نقشه حرارتی دوبعدی (2D Heatmap) از سیاست نهایی استخراج شده، برای نمایش بصری اکشن‌های بهینه در مقابل متغیرهای فضای حالت و مرزهای تصمیم‌گیری عامل.

۵. نمودار میانگین صرفه‌جویی انرژی روش RL در برابر افزایش تعداد Task (۵، ۱۰، ۱۵، ۲۰) برای هر سطح Utilization، جهت ارزیابی اثربخشی نمایش حالت در مدیریت پیچیدگی زمان‌بندی.

سیستم‌های نهفته مدرن نیازمند اجرای هم‌زمان وظایف سخت‌تنابویی (Hard periodic) و رویدادهای نرم‌نامتناوب (Soft aperiodic) هستند. این پروژه یک فریم‌ورک زمان‌بندی ترکیبی را برای تضمین ددلاین وظایف تنابویی و کاهش زمان پاسخ وظایف نامتناوب در پردازنده‌های چندهسته‌ای همگن ارائه می‌دهد. در این ساختار، وظایف تنابویی به صورت ایستا (Offline) با استفاده از روش Partitioned-EDF و الگوریتم Worst-Fit Decreasing (WFD) به هسته‌ها اختصاص می‌یابند. برای استفاده بهینه از ظرفیت باقی‌مانده، وظایف نامتناوب ورودی به صورت پویا (Online) به هسته‌ای که کمترین ددلاین مجازی را از طریق مکانیزم Total Bandwidth Server (TBS) ارائه دهد، تخصیص داده می‌شوند.

شما باید یک شبیه‌ساز رویداد-گسسته (Discrete-event) در Python یا ++C توسعه دهید. در مدل سیستم، وظایف تنابویی با $D_i = P_i$ تعریف شده و با الگوریتم WFD (مرتب‌سازی بر اساس U_i و تخصیص به هسته با بیشترین فضای خالی) پارتیشن‌بندی می‌شوند. هر هسته یک سرور TBS با پهنای باند $U_{s,j} = 1 - U_{periodic,j}$ دارد. با ورود وظیفه نامتناوب، ددلاین مجازی آن روی تمام هسته‌ها محاسبه شده $(V_{d,j} = \max(A_k, V_{d,prev,j}) + E_k/U_{s,j})$ و وظیفه به هسته‌ای با کمترین V_d واگذار می‌گردد. در صورت Preemption، وظایف نامتناوب می‌توانند با ارزیابی مجدد ددلاین‌های مجازی، بین هسته‌ها مهاجرت (Migration) کنند.

برای ارزیابی فریم‌ورک، باید حداقل ۱۰۰ مجموعه وظیفه منحصر به فرد برای هر پیکربندی شبیه‌سازی و میانگین‌گیری شود. وظایف تنابویی با الگوریتم UUniFast و دوره‌هایی از مجموعه $\{10, 20, 50, 100, 200, 500, 1000\}$ میلی‌ثانیه برای $U_p \in \{0.4, 0.5, 0.6\}$ تولید شوند. وظایف نامتناوب نیز با فرآیند پواسون و بار U_a از ۰.۰۵ تا ۰.۴۰ تولید گردند. سیستم باید روی هسته‌های $m \in \{2, 4, 8\}$ و تعداد وظایف N بین 2m تا 10m ارزیابی و با سیستم Baseline (زمان‌بندی Background) مقایسه شود.

نتایج شبیه‌سازی باید در قالب ۵ نمودار زیر تحلیل و ارائه شود:

۱. نمودار زمان پاسخ نرمال‌سازی شده وظایف نامتناوب در مقابل تعداد وظایف (۱۰ تا ۴۰) برای تعداد هسته‌های مختلف (2, 4, 8)

۲. مقایسه عملکرد متد TBS و Background Scheduling با افزایش بار نامتناوب از ۰.۰۵ تا ۰.۴۰ در یک سیستم ۴ هسته‌ای

۳. نمودار ستونی توزیع بهره‌وری کل (تنابویی + نامتناوب) برای هر ۴ هسته در مقابل افزایش بار U_a ، جهت اثبات خاصیت توازن بار (Load-balancing) در تخصیص جهانی TBS.

۴. نمودار نرخ خطای ددلاین وظایف تنابویی در تمامی بارهای U_a که باید دقیقاً روی ۰٪ باشد تا اثبات شود ادغام TBS باعث نقض محدودیت‌های Hard Real-time نمی‌شود.

۵. نمودار میانگین دفعات مهاجرت وظایف نامتناوب در مقابل بار کل سیستم $(U_p + U_a)$ برای تحلیل بار اضافی (Overhead) ناشی از بازتخصیص پویا در زمان Preemption.

در سامانه‌های بحرانی مانند سیستم‌های کنترل پرواز، بروز خطاهای گذرا می‌تواند عواقب فاجعه‌باری داشته باشد. این پروژه به طراحی و ارزیابی مکانیزم‌های کلاسیک تحمل‌پذیری خطا در یک سیستم بی‌درنگ چند هسته‌ای می‌پردازد. چالش اصلی، انتخاب راهکاری است که بتواند ضمن بازیابی سیستم از خطا، کمترین تأثیر منفی را بر وظایف دیگر داشته باشد و از پدیده‌ای به نام آسیب جانبی که در آن وظایف سالم مهلت زمانی خود را از دست می‌دهند، جلوگیری کند. هدف شما، پیاده‌سازی و مقایسه کمی سه رویکرد اصلی جرای مجدد (Re-execution)، چک‌پوینت‌گیری (Checkpointing/Rollback)، و افزونگی فعال (Active Replication)، و تحلیل تأثیر سیاست‌های نگاشت بر کارایی هر یک از این روش‌ها است.

برای شبیه‌سازی، ابتدا با الگوریتم UUnifast-Discard مجموعه وظایف را تولید کنید و دوره تناوب آن‌ها را به صورت تصادفی از مجموعه گسسته $\{10, 20, 30, 40, 50\}$ انتخاب کنید. سپس وظایف را با استفاده از الگوریتم زمان‌بندی Partitioned EDF و با دو سیاست نگاشت متفاوت Best Fit Decreasing (BFD) و Worst Fit Decreasing (WFD) بر روی هسته‌ها قرار دهید. خطاهای گذرا با یک نرخ احتمال مشخص در هر لحظه از اجرای وظایف به سیستم تزریق می‌شوند. یک مکانیزم بازبینی در هر لحظه صحت عملکرد وظیفه را کنترل کرده و در صورت تشخیص خطا، یکی از سیاست‌های تحمل‌پذیری خطا فعال می‌شود.

شما باید سه مکانیزم اصلی تحمل‌پذیری خطا را پیاده‌سازی و در کنار یک سناریوی پایه، مقایسه کنید. سناریوی پایه فاقد هرگونه مکانیزم بازیابی است. مکانیزم اول، اجرای مجدد کامل (Full Re-execution) است که در آن وظیفه‌ای که با اشکال روبرو شده به طور کامل از ابتدا اجرا می‌شود. مکانیزم دوم، چک‌پوینت‌گیری و بازگشت است. در این روش، در طول اجرای هر وظیفه در فواصل زمانی مشخص، وضعیت آن به عنوان یک چک‌پوینت ذخیره می‌شود که خود دارای سربار زمانی است. در صورت بروز خطا، وظیفه به آخرین چک‌پوینت معتبر بازگشته و اجرا را از آن نقطه ادامه می‌دهد؛ چالش اصلی در این روش یافتن فاصله مناسب بین چک‌پوینت‌هاست. مکانیزم سوم، افزونگی فعال است که در آن دو نسخه از هر وظیفه به صورت موازی اجرا می‌شوند و در صورت خطای نسخه اصلی، نتیجه نسخه پشتیبان بلافاصله مورد استفاده قرار می‌گیرد.

برای ارزیابی جامع و دقیق، شبیه‌سازی خود را برای طیف گسترده‌ای از پارامترهای سیستمی و محیطی اجرا کنید. تعداد هسته‌ها (M) را در سه سطح ۲، ۴ و ۸ و تعداد وظایف دوره‌ای را در سطوح ۲، ۳ و ۴ برابر M در نظر بگیرید. برای هر ترکیب، بهره‌وری کل سیستم را در سه حالت $M \cdot 0.5$ ، $M \cdot 0.7$ و $M \cdot 0.9$ بررسی کنید. جهت تحلیل تأثیر نرخ خطا بر عملکرد هر مکانیزم، تمام سناریوها را با سه احتمال خطای گذرا متفاوت ۰.۲٪، ۱٪ و ۵٪ اجرا نمایید. هر اجرای شبیه‌سازی باید به مدت ۱۰ برابر بزرگترین ابر دوره ادامه یابد و برای حصول نتایج آماری معتبر، خروجی‌ها باید میانگین ۱۰۰ اجرا بر روی ۱۰۰ مجموعه وظیفه مستقل باشند. در نهایت نرخ موفقیت وظایف و تعداد وظایفی که با اشکال مواجه شده‌اند را برای هر روش در قالب نمودارهایی تحلیل کنید.

واقعیت مجازی یکی از نمونه‌های واقعی کاربرد سامانه‌های نهفته است. به طور خاص عینک‌های واقعیت مجازی بر پایه این سامانه‌ها طراحی شده‌اند و اغلب از محدودیت‌هایی مانند کمبود قدرت پردازشی یا محدود بودن منابع انرژی رنج می‌برند. در این پروژه شما باید یک نمونه ساده‌ی تک کاربره از چنین سیستمی را شبیه سازی کنید.

سامانه شما دارای یک لپ‌تاپ به عنوان سرور لبه خواهد بود که تمامی اطلاعات محیط سه بعدی را در خود جای داده است. همچنین یک عینک هوشمند فرضی متصل به این لپ‌تاپ وجود دارد که جهت گیری چرخش سر کاربر را دریافت کرده و تصاویر را به وی نمایش می‌دهد. این عینک با ارتباط بیسیم از نوع وایفای و با نرخ 10 MB/s با لپ‌تاپ ارتباط برقرار می‌کند. در هر لحظه از پردازش لپ‌تاپ موظف است که به ازای هر شیء سه بعدی موجود در محیط تصمیم‌گیری کند که آیا به صورت محلی پردازش شده و تصویر آن به کاربر ارسال شود یا اینکه پردازش آن شیء در خود عینک انجام شود. در صورت تصمیم‌گیری برای پردازش در عینک در صورتی که آن شیء در عینک قرار نداشته باشد باید ابتدا به آن از طرف سرور ارسال شود. به منظور مدیریت حافظه‌ی پردازشی عینک دو الگوریتم LRU و LFU مورد استفاده خواهد بود. هر فریم تنها ۲۳ میلی‌ثانیه فرصت پردازش کامل داشته و در صورتی که پردازش ناقص بماند آن فریم از دست رفته حساب می‌شود. هدف در این پروژه آن است که تعداد فریم‌های از دست رفته در طول زمان کمینه شود.

واحد تصمیم گیر

واحد تصمیم‌گیری خودتان را ارائه دهید. واحد تصمیم‌گیری شما باید در اکثر مواقع عملکرد بهتری از سایر الگوریتم‌های مورد مقایسه داشته باشد. برای به روزرسانی حافظه عینک می‌توانید از روش ابتکاری خودتان استفاده کرده یا به سراغ LRU و LFU بروید. به ازای تعداد مختلف اشیاء آزمایش انجام شود.

پارامترها

توان پردازشی و حافظه عینک	توان پردازشی لپ‌تاپ	محدوده تعداد رأس هر شیء	هزینه پردازش رأس‌ها به ازای هر شیء	هزینه پردازش پیکسل‌ها به ازای هر فریم
250 MB - 100 MHz	500 MHz	Uniform[1,40] K	Uniform[100,600] cycle/vertex	Uniform[5,50] cycle/pixel
ابعاد صفحه	حجم هر شیء سه بعدی	نرخ تعداد کل پیکسل‌های اشغالی در هر فریم	تعداد اشیاء محیط	
2064 × 2208	Uniform[0.1,15]MB	Uniform[0,0.5]	۳۰-۲۰-۱۵-۱۰	
		تعداد بیت لازم برای ارسال هر پیکسل		
		۴۰ بیت		

الگوریتم‌های مورد مقایسه

- همواره پردازش در لپ‌تاپ
- همواره پردازش در عینک و به روزرسانی با LRU
- همواره پردازش در عینک و به روزرسانی با LFU
- کاملاً رندوم

توجه: دقت داشته باشید که در هر فریم تعداد پیکسل‌های اشغالی توسط هر شیء به صورت رندوم از مجموعه کل تعداد پیکسل‌های اشغال شده در آن فریم انتخاب شود.

یکی از مدل های وظیفه پر کاربرد در سیستم های نهفته مدل وظیفه ی Directed Acyclic Graph (DAG) است. این مدل از وظیفه وابستگی بین وظایف و ترتیب بین آن ها را مدل می کند. با این حال چیزی که در این نوع مدل بررسی نمی شود احتمال حضور منابع مشترک است، به صورتی که ذاتاً بعضی از گره های موازی ممکن است که نتوانند همراه با هم اجرا شوند چرا که به یک منبع مشترک مانند TPU وابسته اند.

گراف وظایف

برای مدل سازی گراف وظایف از تولید گراف از الگوریتم Task Graphs For Free (TGFF) استفاده خواهد شد. به طور دقیق تر در این پروژه از نسخه 3.6 این ابزار که در گیتهاب قابل دسترسی است استفاده خواهد شد. به طور کلی این الگوریتم فقط پارامترهای تعداد گره (n)، حد بالای درجه خروجی (od) و حد بالای درجه ورودی (id) را دریافت می کند. گراف ها به صورت تک source ایجاد شده و حجم وظایف هر گره به صورت تصادفی بین بازه ی $comp=[1,100]$ واحد انتخاب شده و میزان هزینه ارتباطی بین گره ها نیز به صورت تصادفی از بازه ی حاصل از ضرب پارامتر ccr در comp انتخاب می شود. پس از دریافت گراف از این ابزار شما باید تعداد η برابر تعداد کل گره ها را به صورت رندوم انتخاب کرده و آن ها را با احتمال p به یک دیگر به صورت غیر جهت دار وصل کنید. این یال های جدید نشان دهنده منابع مشترک هستند.

الگوریتم زمانبندی

در ابتدا باید گراف وظایف را به صورت دو لایه جداگانه تعریف کنید. یک لایه، لایه وابستگی بوده و لایه دیگر لایه منابع مشترک است. این دو لایه به یک muxGNN ارائه می شود تا embedding های مناسب برای الگوریتم DRL از آن استخراج شوند. به صورت کلی muxGNN دو مدل GAT را به صورت مستقل بر روی هر لایه گراف پیاده کرده و نتایج را با یک تابع تجميع تبدیل به دو دسته embedding عمیق تر می کند. این دو دسته embedding با هم ترکیب شده همراه با اطلاعات پردازنده ها و به یک واحد masked PPO برای تصمیم گیری ارائه خواهد شد. واحد تصمیم گیر در نهایت مشخص می کند که کدام گره از گراف در کدام هسته از سیستم نهفته اجرا شود. هدف این است که در نهایت زمان اجرای کلی گراف (ماکسیمم زمان اجرای گره های پایانی) کمینه شود. واحد muxGNN به صورت آفلاین توسط یک ساختار auto encoder آموزش داده می شود.

الگوریتم های مورد مقایسه

- رتبه بندی بر اساس مسیر بحرانی و نگاشت بر اساس
 - Worst Fit Decreasing
 - First Fit Decreasing
- انتخاب گره و نگاشت به صورت رندوم

در هر کدام از الگوریتم های فوق در صورت بروز استفاده همزمان از منبع مشترک گرهی مشکل ساز به اولین زمان ممکن در آینده موکول شود.

آزمایش ها و نتایج مورد انتظار

در ابتدا تعداد 30 گراف از تمامی ترکیب های ممکن از پارامترهای جدول زیر تولید کنید.

ρ	η	ccr	od	id	n
0.5	{0.05,0.1,0.2}	{0.2,0.5,1,2}	{2,4}	{2,4}	{10,20,30,40,50}

سپس به ازای شرایط زیر نمودار میانگین زمان پایان را رسم کنید:

- به ازای تعداد هسته ی {4,8,12,16,20} و با پارامترهای: $n=50, id=2, od=2, ccr=0.5, \eta=0.1, \rho=0.5$
- به ازای افزایش n و با تعداد هسته ی 8 و پارامترهای: $id=2, od=2, ccr=0.5, \eta=0.1, \rho=0.5$
- به ازای افزایش id و با تعداد هسته ی 8 و پارامترهای: $n=50, od=2, ccr=0.5, \eta=0.1, \rho=0.5$
- به ازای افزایش od و با تعداد هسته ی 8 و پارامترهای: $n=50, id=2, ccr=0.5, \eta=0.1, \rho=0.5$
- به ازای افزایش ccr و با تعداد هسته ی 8 و پارامترهای: $n=50, id=2, od=2, \eta=0.1, \rho=0.5$
- به ازای افزایش η و با تعداد هسته ی 8 و پارامترهای: $n=50, id=2, od=2, ccr=0.5, \rho=0.5$

پروژه ۲۵	
دستیار آموزشی: محمدهادی کشاورزی	
عنوان:	شبیه ساز نمایشگر طیف صدا
نوع پروژه:	شبیه سازی

کار با صدا یکی از کاربرهای مهم و قدیمی سیستم‌های نهفته است. در گذشته این عملیات با مدارات آنالوگ پیچیده اجرا می‌شد ولی امروزه با استفاده از بردهای دیجیتال این کار به راحتی قابل اجرا است. در این پروژه قصد داریم تا با شبیه سازی یک میکروفرن دیجیتال و یک برد آردوینو یک نمایشگر طیف فرکانس صدا بسازیم. این سامانه دارای حداقل ۱۰ عدد LED خواهد بود که هر یک نمایانگر یک محدوده از طیف فرکانسی قابل دریافت توسط میکروفون هستند. آردوینو به صورت برخط سامانه صدای دریافتی را تحلیل کرده و انرژی هر محدوده فرکانسی را محاسبه و متناسب با مقدار دریافتی شدت نور LED مربوطه را تعیین می‌کند.

نکات پیاده سازی و راهنمایی:

- برای تغییر شدت نور LED باید از قابلیت PWM (Pulse Width Modulation) در ESP32 استفاده کنید.
- از آنجایی که تغییر شدت نور LED رفتار کاملاً خطی ندارد به منظور نمایش بهتر شدت صدا از الگوی نمایی استفاده کنید. دقت کنید که در نمایش شدت هدف دقت بالا نیست ولی باید عملکرد صحیح و تشخیص درست طیف فرکانسی مشهود باشد.
- از مفهوم FFT (Fast Fourier Transform) در استخراج طیف صدا استفاده کنید.
- از شبیه ساز پروتئوس و ورودی دادن فایل صدا استفاده کنید.

با حوزه جدیدی از پروژه های امروزه به سراغ مفهوم دو قولوی دیجیتال یا همان Digital Twin رفته اند. ایده کلی این حوزه این است که علاوه بر پارامترهای فیزیکی یک محیط، یک شبیه سازی دیجیتال و همزمان از محیط نیز داشته باشیم تا بتوان عمل کرد سامانه را به طور همزمان مشاهده کرده و در صورت نیاز رفتار سامانه را نسبت به پارامترهای محیط بررسی کنیم. این امر کمک می کند که علاوه بر ارائه ظاهری مطلوب به کاربر نهایی بتوان رفتار سامانه را در مقابل اتفاقات و سناریوهای احتمالی پیش بینی کرد. در این پروژه قصد داریم که با استفاده از محیط شبیه ساز یک نمونه صنعتی از این مفهوم را پیاده سازی کنیم.

گام اول

در اولین گام باید با استفاده از زبان برنامه نویسی دلخواه خود یک محیط چهار راه را پیاده سازی کنید. این چهار راه دارای یک چراغ راهنمایی عبور خودرو و یک چراغ عابر پیاده در هر ورودی خیابان بوده و ویژگی های زیر را باید داشته باشد:

۱. در صورت روشن بودن چراغ سبز یا زرد برای اتومبیل ها چراغ قرمز برای عابر پیاده متناظر روشن باشد.
۲. در صورتی که چراغ قرمز برای اتومبیل ها روشن است. چراغ سبز برای عابر پیاده متناظر روشن شود.
۳. همواره یکی از چراغ های مخصوص عابر و یکی از چراغ های مخصوص اتومبیل روشن باشد
۴. مدت زمان روشن بودن چراغ قرمز و سبز برای اتومبیل ها 3 دقیقه بوده و مدت زمان روشن بودن چراغ زرد یک دقیقه باشد.

گام دوم

در مفهوم دوقلوی دیجیتال رابط کاربری از اهمیت بالایی برخوردار است. به همین منظور پس از پیاده کردن منطق چهار راه فوق حتما باید یک نمایش گرافیکی از آن را به کاربر ارائه دهید. این نمایش به منظور cross platform بودن و را حتی در دسترسی کاربر باید تحت وب پیاده شده باشد. برای سادگی در این پروژه از Local Host سیستم خود استفاده کنید.

گام سوم

تا الان یک سیستم مانیتورینگ ساده ولی کارآمد را طراحی کرده اید. کار دیگری که در دوقلوی دیجیتال انجام می شود پیش بینی و تغییر پارامترهای محیطی است. در این گام می خواهیم تست بکنیم که اگر سنسور تشخیص حضور خودرو به سیستم اضافه کنیم چه تأثیری بر کل سامانه داشته و چگونه به ما در بهینه شدن سیستم کمک می کند.

پیاده کردن یک نمونه فیزیکی از همان ابتدا معمولاً هزینه بر بوده و تأثیر آن تا قبل از پیاده سازی کامل مشخص نمی شود. به همین منظور از شما می خواهیم که در هر چراغ قرمز یک سنسور تشخیص حضور خودرو (سنسور مغناطیسی با خروجی باینری 0 یا 1 برای حضور یا عدم حضور خودرو) اضافه کنید. حال منطق عملکردی چهار راه را به گونه ای تغییر دهید که میانگین معطلی هر خودرو به کمتر از یک دقیقه برسد.

ویژگی های رفتاری چهار راه به شرح زیر هستند:

- در این حالت توزیع ورود ماشین به هر چراغ را به صورت توزیع پواسون و با مقادیر میانگین 1، 1.5، 1 و 0.5 ماشین بر دقیقه فرض کنید.
- در هر لحظه فقط یک چراغ می تواند سبز باشد.
- عبور هر خودرو از چهار راه 15 ثانیه طول کشیده و این تأخیر برای خودرو های عقب تر در صف چراغ قرمز به صورت تجمعی محاسبه می شود
- دقت کنید که ارائه ظاهری مناسب از اهمیت بالایی دارد.

دسترسی به حافظه از مهم‌ترین عوامل تاثیرگذار بر روی زمان اجرای وظایف سامانه‌های نهفته است. در این پروژه شما میبایست تاثیر حجم داده در زمان اجرای و miss ratio سطوح مختلف حافظه cache را بررسی کنید. برای این بررسی مراحل زیر را طی کنید.

۱. با استفاده از زبان برنامه نویسی C ابتدا یک فایل ورودی که حاوی تعدادی اعداد بین ۰ تا ۶۴ است را خوانده و سپس ۱۰۰۰۰۰ دسترسی تصادفی با توزیع یکنواخت بر روی آن اعمال کنید. فایل‌های ورودی و تعداد اعداد هر فایل باید مطابق جدول زیر باشد.

۲. کد C را برای ARM کامپایل کنید و در gem5 یک پردازنده ARM Cortex A7 با مشخصات زیر ایجاد کرده و کد کامپایل شده خود را بر روی آن اجرا کنید. (استفاده از هر یک از حالت‌های syscal و full system شبیه ساز gem5 منعی ندارد)

Input File	Number of inputs
1	5000
2	50000
3	150000
4	500000
5	1200000

Parameter	Value
RAM	512MB
L1 Instruction Cache (L1I)	32KB
L1 Data Cache (L1D)	32KB
L2 Data Cache (L2D)	512KB

خروجی‌ها:

- نمودار نرخ نقصان حافظه نهان سطح ۱ به ازای هر یک از فایل‌های ورودی
- نمودار نرخ نقصان حافظه نهان سطح ۲ به ازای هر یک از فایل‌های ورودی
- زمان اجرا به ازای هر یک از فایل‌های ورودی
- تحلیل نمودارها و نتایج بدست آمده

نکته: پیاده‌سازی تمامی موارد گفته‌شده، تولید تمام خروجی‌های خواسته‌شده و تهیه گزارش پایانی برای این پروژه الزامی است. در گزارش پایانی می‌بایست کدهای پیاده‌سازی شده خط به خط توضیح داده شوند و نتایج بدست آمده بصورت نمودار در گزارش قرار داده شده و تحلیل شوند.

پروژه ۲۸	
دستیار آموزشی: صادق خدری	
عنوان:	جایابی بخش‌های یک تراشه برای حداقل سازی دمای حالت گذرا
نوع پروژه:	شبیه‌سازی

با این پروژه به بررسی تاثیر جایابی بخش‌های مختلف یک تراشه سامانه نهفته در حداکثر دمای حالت گذرای آن اختصاص دارد. برای انجام این پروژه مراحل زیر باید طی شود.

۱. چهار floorplan با جایابی‌های متفاوت از یک تراشه با بخش‌های زیر ایجاد کنید. هر یک از این جایابی‌ها باید منطقی و قابل توجیه باشد.

Component	Width (mm)	Height (mm)	Area (mm2)
CPU Core 0	2	2	4
CPU Core 1	2	2	4
NPU (AI Accel)	3	3	9
GPU	4	3	12
L3 Cache	6	2	12
SRAM Shared	3	4	12

۲. یک فایل power trace برای کامپوننت‌های تراشه با ویژگی‌های زیر ایجاد کنید.

Time Step (ms)	CPU_0	CPU_1	NPU	GPU	L3_Cache	SRAM_Shared
1 – 20	0.2	0.2	0.1	0.5	0.8	0.5
21 – 70	6	6	15	8	1.2	0.8
71 – 100	0.4	0.4	0.2	1	0.8	0.5

۳. در مرحله بعد با در نظر گرفتن تنظیمات زیر در شبیه‌ساز hotspot حداکثر دمای حالت گذرا را بدست آورید.

Parameter	Value	Parameter	Value
model_type-	Block	c_convex-	140.4
t_ambient-	300.15	chip_thickness-	0.000015
-init_temp	300.15	s_sink-	0.06
-sampling_intvl	0.001	t_sink-	0.0069
-r_convex	0.1	-s_spreader	0.03
-t_spreader	0.001	-detailed_3D	off

۴. حداکثر دمای حالت گذرای هر یک از floorplanها را تحلیل و بررسی کنید.

خروجی‌ها:

- نمودار مقایسه دمای حالت گذرای floorplanها
- تحلیل حداکثر دمای حالت گذرا
- تحلیل تاثیر جایابی در حداکثر دمای حالت گذرا

نکته: پیاده‌سازی تمامی موارد گفته‌شده، تولید تمام خروجی‌های خواسته‌شده و تهیه گزارش پایانی برای این پروژه الزامی است. در گزارش پایانی می‌بایست کدهای پیاده‌سازی شده خط به خط توضیح داده شوند و نتایج بدست آمده بصورت نمودار در گزارش قرار داده شده و تحلیل شوند.

پروژه ۲۹

دستیار آموزشی: صادق خدری

عنوان:

ارزیابی تاثیر بخش‌های مختلف پردازنده در توان مصرفی و مساحت

نوع پروژه:
شبیه‌سازی

هر پردازنده دارای اجزا مختلفی شامل هسته‌های پردازشی، حافظه نهان، ارتباطات مخلف و غیره است. در این پروژه به بررسی تاثیر جزئیات بخش‌های مختلف پردازنده در توان مصرفی و مساحت می‌پردازیم و برای این کار از شبیه‌ساز McPAT استفاده می‌شود. برای این پروژه فایل ARM_A9_2GHz استفاده کرده و تغییر مقادیر زیر را بررسی کنید.

ردیف	اجزا	مقدار پیشفرض	مقادیر مورد بررسی
۱	Technology Node	40 nm	40, 32, 22, 12 nm
۲	Core Frequency	2000 MHz	500, 1000, 1500, 2000 MHz
۳	ROB Size	64	32, 64, 96
۴	Integer ALUs	2	1, 2, 3
۵	FP Units	1	0, 1, 2
۶	L1 Instruction Size	32 KB	16, 32, 64 KB
۷	L1 Data Size	32 KB	16, 32, 64 KB
۸	Clock Gating	1	0, 1
۹	Instruction Window	32	16, 32, 64

مقادیری که در جدول فوق ذکر نشده است را به صورت معقول و متناسب با پردازنده ذکر شده در نظر گرفته و خروجی‌های زیر را در گزارش خود ارائه کنید.

خروجی‌ها:

- نمودار مساحت بر حسب هر یک از تغییرات
- نمودار توان مصرفی بر حسب هر یک از تغییرات
- تحلیل نتایج بدست آمده در هر نمودار

نکته: پیاده‌سازی تمامی موارد گفته‌شده، تولید تمام خروجی‌های خواسته‌شده و تهیه گزارش پایانی برای این پروژه الزامی است. در گزارش پایانی می‌بایست کدهای پیاده‌سازی شده خط به خط توضیح داده شوند و نتایج بدست آمده بصورت نمودار در گزارش قرار داده شده و تحلیل شوند.

با در سیستم‌های نهفته، قابلیت اطمینان برای کاربردهای safety critical از اهمیت بسیار بالایی برخوردار است. یکی از فریم‌ورک‌های پرکاربرد در این سامانه‌ها، سیستم‌عامل رباتیک ROS2 است. اگرچه ROS2 یک میان‌افزار قدرتمند برای توسعه سیستم‌های رباتیک به شمار می‌رود، اما به صورت پیش‌فرض فاقد مکانیزم‌های داخلی تحمل‌پذیری خطا است.

یکی از روش‌های متداول برای ایجاد تحمل‌پذیری خطا، استفاده از مکانیزم standby-sparing است. این مکانیزم انواع مختلفی دارد که در این پروژه، تمرکز ما بر روی پیاده‌سازی دو مدل hot spare و cold spare در محیط ROS2 خواهد بود. برای افزودن این قابلیت‌ها به سیستم، به یک ماژول watchdog نیاز است تا توپولوژی سامانه را به صورت مداوم رصد کرده و در صورت بروز مشکل، گره‌های پشتیبان (cold spare) یا (hot spare) را فعال کند. در نهایت، با توجه به مقادیر قابلیت اطمینان مفروض برای گره‌های سیستم، باید از روش مونت‌کارلو برای تعیین قابلیت اطمینان کل سامانه استفاده شود.

- پیاده‌سازی هر دو مدل افزونگی hot spare و cold spare در محیط ROS2
- شبیه‌سازی رخداد خطا در سامانه به گونه‌ای که قابلیت اطمینان هر گره از بازه [۰.۸, ۰.۹, ۰.۹۹, ۰.۹۹۹] انتخاب شود. (برای شبیه‌سازی باید از autoware reference benchmark استفاده نمایید)
- محاسبه قابلیت اطمینان کل سامانه با استفاده از روش مونت‌کارلو و مقایسه آن با نتایج حاصل از پیاده‌سازی عملی بر روی ROS2

نمودارهای ارزیابی مورد نیاز

- (۱) قابلیت اطمینان سیستم در برابر گره: نمایشی از مقایسه قابلیت اطمینان کل سامانه در سه حالت بدون افزونگی، با hot spare و با cold spare به ازای ۵ سطح مشخص‌شده برای قابلیت اطمینان گره‌ها
- (۲) توزیع زمان بازیابی سیستم: نمودار توزیع آماری برای مقایسه زمان سوئیچ و بازیابی از لحظه بروز خرابی تا فعال‌شدن کامل پشتیبان در دو مدل hot spare و cold spare
- (۳) همگرایی مونت‌کارلو: نموداری که نشان می‌دهد چگونه مقدار تخمین‌زده‌شده برای قابلیت اطمینان کل سیستم، با افزایش تعداد دورهای شبیه‌سازی همگرا شده و به یک مقدار پایدار می‌رسد.
- (۴) مقایسه سربار منابع محاسباتی: نمودار میله‌ای جهت مقایسه میزان مصرف پردازنده و حافظه در سه سناریوی مختلف: سامانه عادی (بدون افزونگی)، سامانه با hot spare (پردازش همزمان) و سامانه با cold spare (آماده به کار).
- (۵) تأخیر انتهابه‌انتهای در شرایط بروز خطا: نمودار سری زمانی (Time-Series) برای نمایش تغییرات تأخیر پیام‌ها در مسیرهای بحرانی در لحظه رخداد خطا و بلافاصله پس از سوئیچ روی گره پشتیبان.
- (۶) نرخ موفقیت انتقال: نمودار نمایش‌دهنده درصد موفقیت سامانه در بازیابی خطا بدون از دست دادن پیام‌های حیاتی
- (۷) تحلیل حساسیت زمان تشخیص واچ‌داگ: بررسی و نمایش تأثیر فرکانس پایش واچ‌داگ بر روی قابلیت اطمینان سیستم و زمان بازیابی

در این پروژه، یک سیستم پردازشی چند هسته‌ای ناهمگن (Heterogeneous Multi-core System) مورد بررسی قرار می‌گیرد. این سیستم از هسته‌هایی با توان پردازشی و مصرف انرژی متفاوت تشکیل شده است. وظایف (Tasks) به صورت آنلاین و پویا وارد سیستم می‌شوند. هر وظیفه دارای مشخصاتی نظیر حجم پردازشی، زمان ورود و یک ضرب‌الاجل (Deadline) حیاتی است. هدف اصلی طراحی یک سیستم تصمیم‌گیرنده هوشمند است که به محض ورود هر وظیفه، تعیین کند آن وظیفه روی کدام هسته اجرا شود تا کیفیت خدمات (QoS) به حداکثر رسیده و مصرف انرژی حداقل شود. پس از تخصیص، وظایف در صف هر هسته با استفاده از الگوریتم زمان‌بندی EDF اجرا می‌شوند.

پیچیدگی‌ها و دینامیک سیستم

- ورود تصادفی و متغیر وظایف: نرخ ورود وظایف ثابت نیست و از یک توزیع تصادفی (مانند توزیع پواسون) پیروی می‌کند که باعث ایجاد بار ترافیکی (Burst) لحظه‌ای روی سیستم می‌شود.
 - پشتیبانی از مقیاس‌بندی پویای ولتاژ و فرکانس (DVFS): فرکانس کاری هسته‌ها ثابت نیست. عامل هوشمند علاوه بر انتخاب هسته، می‌تواند فرکانس اجرای آن هسته را نیز برای مدیریت مصرف انرژی ($E \propto V^2 f$) تغییر دهد.
 - محدودیت‌های حرارتی (Thermal Constraints): استفاده مداوم از هسته‌های با کارایی بالا باعث افزایش دمای سیستم می‌شود که در صورت عبور از آستانه مجاز، فرکانس هسته به صورت خودکار کاهش می‌یابد (Thermal Throttling). این دینامیک نیازمند مدیریت هوشمندانه توزیع بار است.
- رویکرد حل مسئله و مدل‌سازی DRL:** مسئله تخصیص وظایف به عنوان یک فرآیند تصمیم‌گیری مارکوف (MDP) مدل‌سازی می‌شود. با توجه به پیچیدگی و پیوستگی فضای حالت، دانشجویان باید از مدل‌های مختلف یادگیری تقویتی عمیق (DRL) استفاده کرده و آن‌ها را پیاده‌سازی و مقایسه کنند.

الگوریتم‌های مورد نیاز برای مقایسه:

۱. الگوریتم DDQN (DoubleDeepQ – Network)

۲. الگوریتم PPO (Proximal Policy Optimization)

چالش تأخیر تصمیم‌گیری (Inference Overhead): از آنجایی که اجرای مستقیم شبکه‌های عصبی عمیق به ازای ورود هر وظیفه در یک سیستم بیدرنگ باعث ایجاد تأخیر محاسباتی سنگینی می‌شود، رویکرد پروژه به شکل زیر است:

- فاز یادگیری (آفلاین): ابتدا مدل‌های DRL (مانند PPO و DDQN) در یک محیط شبیه‌سازی آموزش داده می‌شوند تا سیاست بهینه (تخصیص وظیفه به هسته مناسب) را بیاموزند. سپس از این مدل آموزش‌دیده، برای برچسب‌گذاری (Labeling) یک مجموعه داده بزرگ از حالت‌های مختلف سیستم استفاده می‌شود.
- فاز اجرا (آنلاین): برای تصمیم‌گیری سریع و بیدرنگ، از روش‌های دسته‌بندی (Classification) با سربار محاسباتی بسیار پایین (مانند SVM یا معیار "فاصله تا میانگین هر دسته") استفاده می‌شود که بر روی داده‌های برچسب‌گذاری‌شده توسط عامل DRL آموزش دیده‌اند.

خروجی‌ها و نیازمندی‌های ارزیابی: دانشجویان باید شبیه‌سازی را انجام داده و خروجی‌های زیر را استخراج و تحلیل کنند:

- مقایسه عملکرد الگوریتم‌های DRL: رسم نمودار همگرایی برای مدل‌های DDQN، PPO و SAC و بررسی اینکه کدام یک در ایجاد داده‌های برچسب‌گذاری شده عملکرد بهتری دارد.
- مقایسه سه سناریوی تخصیص آنلاین: مقایسه عملکرد سیستم در سه حالت تخصیص با روش دسته‌بندی پیشنهادی (فاصله تا میانگین یا SVM)، تخصیص با اجرای مستقیم مدل DDQN/PPO (با در نظر گرفتن تأخیر اجرا)، و تخصیص تصادفی.
- نمودارهای ارزیابی نهایی:
 - مقایسه کیفیت خدمات (QoS) (میزان وظایف از دست رفته به دلیل رد شدن از ضرب‌الاجل).
 - مقایسه انرژی مصرف شده برای کل سیستم.
 - نمودار Gantt Chart اجرای وظایف روی هسته‌های مختلف بر اساس الگوریتم EDF و نمایش توزیع ویژگی‌های وظایف اختصاص یافته روی هر هسته.

با در این پروژه، یک سامانه محاسبات لبه (Edge Computing) شامل یک Edge Server قدرتمند و تعداد زیادی کاربر نهایی (User Equipment) با توان پردازشی محدود بررسی می‌شود. به دلیل حجم بالای کاربران، از یک ساختار سلسله‌مراتبی (Hierarchical) استفاده می‌شود؛ به این معنا که کاربران در ابتدا بر اساس موقعیت مکانی و توانمندی‌هایشان به صورت پویا خوشه‌بندی می‌شوند.

در هر بازه زمانی (Time Slot)، یک وظیفه (Task) جدید برای هر کاربر تولید می‌شود. هر وظیفه دارای دو ویژگی حیاتی است: تعداد چرخه‌های پردازشی مورد نیاز (Workload) و ضرب‌الاجل (Deadline). در این سیستم، تصمیم‌گیری باید برای دو چالش اساسی انجام شود:

(۱) Offloading Decision: هر وظیفه به صورت محلی روی دستگاه کاربر اجرا شود یا برای پردازش به سرور لبه ارسال گردد.

(۲) Resource Allocation: منابع محاسباتی و پهنای باند سرور چگونه بین وظایف ارسال شده توزیع شود.

وظایفی که به سرور لبه منتقل می‌شوند، در صف اجرا قرار گرفته و توسط الگوریتم EDF (Earliest Deadline First) زمان‌بندی می‌شوند.

تحرك‌پذیری کاربران (Mobility): کاربران دارای حرکت تصادفی در محیط هستند. این حرکت باعث تغییر پیوسته فاصله آن‌ها از Access Point شده و در نتیجه نرخ انتقال داده (Data Rate)، تأخیر شبکه و انرژی مصرفی برای ارسال وظایف در هر لحظه متغیر و دارای نویز خواهد بود.

ظرفیت متغیر سرور: منابع پردازشی در دسترس سرور لبه ثابت نیست و در هر Time Slot به دلیل پردازش‌های پس‌زمینه دارای نوسان (Fluctuation) است.

رویکرد حل مسئله (استفاده از DRL)

دانشجویان باید یک معماری تصمیم‌گیری سلسله‌مراتبی بر پایه الگوریتم‌های یادگیری تقویتی عمیق (Deep Reinforcement Learning) طراحی کنند:

- عامل سطح بالا (High-level Agent): منابع کلی سرور لبه را بین خوشه‌های مختلف توزیع می‌کند.
- عامل‌های سطح پایین (Low-level Agents): درون هر خوشه قرار دارند و وظیفه تصمیم‌گیری Offloading برای تک‌تک کاربران زیرمجموعه خود و توزیع منابع تخصیص‌یافته به خوشه را بر عهده دارند.

تأکید بر ارزیابی مدل‌های مختلف:

دانشجویان موظف‌اند معماری فوق را پیاده‌سازی کرده و برای عامل‌های هوشمند، حتماً از الگوریتم‌های مختلف خانواده DRL استفاده نموده و عملکرد آن‌ها را با یکدیگر مقایسه کنند. مدل‌های مورد انتظار برای پیاده‌سازی و مقایسه عبارتند از:

- DQN و Double DQN (DDQN) (برای فضای عمل گسسته)
- PPO (Proximal Policy Optimization)

خروجی‌ها و نیازمندی‌های پیاده‌سازی

شبکه باید برای جمعیتی معادل ۱۰۰ کاربر و ۴ الی ۱۰ خوشه شبیه‌سازی شود. در پایان، عملکرد الگوریتم‌های مختلف (DDQN, PPO و ...) باید با یکدیگر مقایسه شده و خروجی‌های زیر استخراج گردند:

- (۱) نمودار همگرایی مجموع Quality of Service (QoS) در طول فرایند آموزش برای مدل‌های مختلف DRL.
- (۲) نمودار مجموع انرژی مصرفی کل سیستم در طول یادگیری.
- (۳) نمودار مقایسه‌ای QoS و مصرف انرژی به تفکیک خوشه‌ها.
- (۴) نمایش بصری نحوه زمان‌بندی و چیدمان وظایف واگذار شده روی سرور لبه توسط الگوریتم EDF در یک Time slot مشخص.
- (۵) تحلیل تأثیر تحرك کاربران (تغییر پهنای باند) و نوسانات ظرفیت سرور بر پایداری مدل‌های مختلف DRL.

با این پروژه با هدف ارزیابی دقیق سطح معماری و تحلیل توان مصرفی پردازنده‌های تعبیه‌شده (Embedded Systems)، به طور خاص معماری Cortex-M4، تعریف شده است. در گام نخست، شبیه‌ساز معماری gem5 به منظور مدل‌سازی دقیق ویژگی‌های سخت‌افزاری این پردازنده پیکربندی می‌گردد. از آنجا که این پردازنده فاقد واحد مدیریت حافظه (MMU) است، از مدل پردازشی درون‌ترتیب (In-order) همراه با پیکربندی حافظه و خط لوله (Pipeline) متناسب با معماری ARMv7E-M استفاده خواهد شد تا رفتار سخت‌افزار با بالاترین دقت ممکن شبیه‌سازی شود.

در گام دوم، شبیه‌سازی سیستم کامل (Full System) برای اجرای برنامه‌های محک صورت می‌پذیرد.

پس از اتمام اجرای برنامه‌های محک، خروجی‌های آماری شبیه‌ساز gem5 به منظور استخراج سربار سخت‌افزاری و معیارهای کارایی مورد تحلیل قرار می‌گیرند.

در گام نهایی، به منظور محاسبه توان مصرفی و مساحت مصرفی تراشه، داده‌های آماری و پیکربندی استخراج‌شده از gem5 با استفاده از اسکریپت‌های مبدل به فرمت استاندارد XML برای ابزار McPAT تبدیل می‌شوند. با اجرای McPAT، پروفایل توان مصرفی هر یک از برنامه‌های محک به تفکیک توان استاتیک و دینامیک به دست آمده و در نهایت، یک تحلیل جامع از تقابل کارایی در برابر توان مصرفی (Performance/Power Trade-off) برای معماری Cortex-M4 ارائه خواهد شد.

با سیستم‌های نهفته (Embedded Systems) در کاربردهای صنعتی و حساس، نیازمند سطح بالایی از قابلیت اطمینان و در دسترس بودن هستند. یکی از چالش‌های اساسی در این سیستم‌ها، توقف یا خرابی پردازش‌های حیاتی به دلیل خطاهای نرم‌افزاری یا شرایط پیش‌بینی‌نشده است. در این پروژه تحقیقاتی، یک مکانیزم تحمل خطا (Fault Tolerance) مبتنی بر نظارتگر نرم‌افزاری (Software Watchdog) به صورت یک ماژول قابل بارگذاری هسته لینوکس (LKM) طراحی و پیاده‌سازی می‌گردد. هدف اصلی این ماژول، پایش مستمر پردازش‌های سطح کاربر (User-space) و بازیابی سریع آن‌ها در صورت بروز خطا است، به طوری که نیاز به راه‌اندازی مجدد و زمان‌بر کل سخت‌افزار (Hardware Reset) برطرف گردد.

معماری این سیستم بر پایه ارتباط مستمر بین فضای کاربر و فضای هسته بنا شده است. برنامه کاربردی در بازه‌های زمانی مشخص، سیگنال‌های حیات (Heartbeat) را از طریق یک رابط در سیستم‌فایل مجازی (مانند procfs) به ماژول هسته ارسال می‌کند. در سمت هسته، یک زمان‌سنج (Kernel Timer) وظیفه بررسی این سیگنال‌ها را بر عهده دارد. چنانچه برنامه کاربردی به دلیل بروز بن‌بست (Deadlock) یا افتادن در حلقه‌های بی‌نهایت قادر به ارسال سیگنال در بازه مجاز نباشد، زمان‌سنج منقضی شده و هسته به صورت خودکار با ارسال سیگنال خاتمه به پردازش معیوب، منابع را آزاد و مسیر را برای اجرای مجدد سرویس هموار می‌سازد.

به منظور ارزیابی کارایی و اثبات صحت عملکرد ماژول، یک برنامه کاربردی با قابلیت تزریق خطا (Fault Injection) توسعه می‌یابد. این برنامه پس از اجرای موفقیت‌آمیز در یک دوره زمانی مشخص، به صورت مصنوعی و برنامه‌ریزی‌شده وارد یک وضعیت توقف کامل (مانند یک حلقه بی‌نهایت بی‌قید و شرط) می‌شود. این شبیه‌سازی خرابی باعث قطع شدن سیگنال‌های حیات شده و به محقق اجازه می‌دهد تا واکنش ماژول هسته را در شرایط بحرانی به دقت مورد سنجش قرار دهد.

برای سنجش دقیق و سطح معماری این راهکار، کل سیستم‌عامل به همراه ماژول توسعه‌یافته بر روی شبیه‌ساز دقیق gem5 در حالت سیستم کامل (Full System) اجرا می‌گردد. بهره‌گیری از شبیه‌ساز این امکان را فراهم می‌کند که تاخیر تشخیص و بازیابی بر اساس چرخه‌های پردازشی (Clock Cycles) با بالاترین دقت اندازه‌گیری شود. زمان بازیابی سیستم از طریق رابطه

$$T_{\text{recovery}} = (\text{Cycles_kill} - \text{Cycles_fault}) / F_{\text{cpu}}$$

محاسبه خواهد شد. در نهایت، تاثیر این روش بر ارتقای دسترس‌پذیری سیستم با استفاده از فرمول $A = \frac{MTBF}{MTBF + MTTR}$ تحلیل می‌شود که نشان‌دهنده کاهش چشمگیر میانگین زمان تعمیر (MTTR) نسبت به نظارتگرهای سخت‌افزاری سنتی خواهد بود.

در این پروژه می‌خواهیم مجموعه‌ای از تسک‌ها رو که به هم وابستگی دارن، روی یک سیستم ناهمگون پردازش کنیم. سیستم ما شامل هسته‌های پرقدرت (HP) و هسته‌های کم‌توان (LP) هست. هدف این است که با استفاده از الگوریتم یادگیری تقویتی DDQN یاد بگیریم چطور این تسک‌ها رو بین هسته‌ها پخش کنیم تا هم وابستگی‌های گراف رعایت بشود، هم زمان کل اجرا و مصرف انرژی کم بشود و از همه مهم‌تر، ددلاین نهایی برنامه رد نشه. در نهایت هم می‌خوایم نتایج تئوری رو با یک شبیه‌ساز دقیق سیستم معماری ارزیابی کنیم.

مدل سامانه و وظایف

یک سیستم 4 هسته‌ای رو در نظر می‌گیریم:

- 2 هسته HP با فرکانس بالا (fHP) (سریع و پرمصرف).
- 2 هسته LP با فرکانس پایین (fLP) (کند و کم‌مصرف).

هر گره v_i تو گراف DAG ما یک تسک با بار پردازشی c_i (تعداد سایکل) هست. زمان اجرای تسک i روی هسته‌ی انتخابی به صورت تئوری از این رابطه به دست میاد:

$$T_i = \frac{c_i}{f}$$

انرژی مصرفی تسک روی همون هسته هم برابر است با:

$$E_i = \omega c_i (f^2)$$

که در اون ω ثابت انرژی هسته‌ست و f فرکانس پردازنده است.

رتبه‌بندی وظایف

قبل از اینکه الگوریتم هوشمند ما بخواد تسک‌ها رو نگاشت کنه، باید تعیین کنیم تسک‌های گراف به چه ترتیبی وارد سیستم بشن. اینجا می‌تونیم از مرتب‌سازی توپولوژیک (Topological Sort) استفاده کنیم تا مطمئن بشی تسکی که انتخاب میشه، حتماً پیش‌نیازهایش قبلاً اجرا شدن.

نگاشت وظایف با الگوریتم DDQN

در این مسئله، سیستم به صورت یک فرایند تصمیم‌گیری مارکوف (Markov Decision Process) مدل‌سازی شده و عامل هوشمند با استفاده از الگوریتم آموزش داده می‌شود. مزیت اصلی DDQN نسبت به DQN متداول، استفاده از دو شبکه مجزا برای انتخاب عمل و ارزیابی مقدار آن است که منجر به کاهش تخمین‌های بیش‌ازحد خوش‌بینانه و در نتیجه، یادگیری پایدارتر می‌شود.

- **فضای حالت (State Space):** فضای حالت می‌تواند شامل متغیرهایی نظیر سیکل پردازشی تسک جاری، زمان در دسترس قرار گرفتن هسته با توان بالا، زمان در دسترس قرار گرفتن هسته با توان پایین و درصد تسک‌های باقیمانده نسبت به کل گراف وظایف باشد.
- **فضای اعمال (Action Space):** مجموعه اعمال شامل دو گزینه است: اجرای تسک روی هسته کم‌مصرف با اکشن 0 و اجرای تسک روی هسته پرقدرت با اکشن 1.
- **طراحی تابع پاداش (Reward Function):** طراحی تابع پاداش بر عهده توسعه‌دهنده است و فرمول از پیش تعیین‌شده‌ای ارائه نمی‌شود. هدف، تعریف تابع پاداشی است که عامل هوشمند را به ایجاد بهترین توازن میان کاهش مصرف انرژی، کاهش زمان کل اجرا (Makespan) و رعایت محدودیت‌های زمانی (Deadline) هدایت کند. در این راستا می‌توان از پناالتی‌ها، پاداش‌های مرحله‌ای یا پاداش نهایی متناسب با عملکرد کلی سیستم استفاده کرد.

برای اینکه پروژه فقط در حد فرمول‌های ساده نمونه، قراره از شبیه‌ساز قدرتمند Gem5 استفاده کنید.

- **بخش آموزش:** می‌تونن عامل DDQN رو با همون فرمول‌های تئوری (Ei و Ti) آموزش بدی تا سریع‌تر همگرا بشود.
- **بخش تست و اعتبارسنجی:** برای گراف‌های تست، بعد از اینکه مدل DDQN تصمیم گرفت هر تسک باید روی کدوم هسته اجرا بشود، این پیکربندی (زمان‌بندی و نگاشت) رو می‌برید در شبیه‌ساز Gem5 (مثلاً با تنظیم یک معماری شبیه ARM big.LITTLE). در Gem5 گرافیک اجرای تسک‌ها رو شبیه‌سازی می‌کنید و آمار واقعی زمان و مصرف انرژی رو از خروجی‌های Gem5 استخراج می‌کنید.

خروجی‌های مورد انتظار:

- **تحلیل تابع پاداش:** نحوه طراحی تابع پاداش به‌صورت دقیق تشریح شود و توضیح داده شود که پارامترهای زمان اجرا و مصرف انرژی چگونه در این تابع متوازن و وزن‌دهی شده‌اند.
- **مقایسه با روش‌های کلاسیک:** عملکرد مدل پیشنهادی مبتنی بر DDQN، بر اساس خروجی‌های شبیه‌ساز Gem5، با روش‌های زیر مقایسه شود:
 - **HPC-Only:** اجرای تمامی تسک‌های گراف بر روی هسته‌های با کارایی بالا.
 - **LPC-Only:** اجرای تمامی تسک‌های گراف بر روی هسته‌های کم‌مصرف.
 - **Random-Mapping:** نگاشت تصادفی تسک‌ها بر روی هسته‌ها.

این پروژه، توسعه یک چارچوب نرم‌افزاری یکپارچه و مبتنی بر یادگیری ماشین جهت پروفایل‌گیری، ارزیابی و پیش‌بینی دقیق توان مصرفی و وضعیت حرارتی سیستم‌های چند هسته‌ای در حین اجرای برنامه‌های محک (Embench) است. با توجه به سربار محاسباتی بسیار بالای ابزارهای تحلیلی توان و حرارت در سطح معماری، این پروژه قصد دارد با اجرای یک زنجیره ابزار آفلاین متشکل از شبیه‌ساز gem5، ابزار مدل‌سازی توان McPAT و شبیه‌ساز حرارتی HotSpot، یک مجموعه داده جامع تولید نماید. در نهایت، با استفاده از این داده‌ها، یک شبکه عصبی بازگشتی (LSTM) در لایه‌ای بالاتر آموزش داده می‌شود تا بتواند به صورت بی‌درنگ (Real-time) و تنها با دریافت رویدادشمارهای سخت‌افزاری، توان مصرفی پویای سیستم را تخمین بزند.

مشابه فرآیندهای استاندارد شبیه‌سازی، ابتدا بارهای کاری استاندارد (Embench) برای معماری هدف کامپایل متقاطع شده و در قالب فایل‌های باینری به درون دیسک ایمج‌های سازگار با gem5 انتقال می‌یابند. این روش ضمن تسریع فرآیند آگازین (Boot) و اجرای برنامه‌ها، امکان مدیریت دقیق‌تر فرآیندهای سیستم‌عامل تعبیه‌شده را در محیط شبیه‌ساز فراهم می‌آورد.

در این بخش، یک اسکریپت کنترل‌کننده (به زبان پایتون) توسعه می‌یابد تا اجرای برنامه‌ها را روی هسته‌های مختلف در قالب ابردوره‌های (Hyper-period) مشخص با نماد H مدیریت کند. هر ابردوره به بازه‌های زمانی خرد Δt تقسیم می‌شود. در پایان هر بازه Δt ، آمار اجرای gem5 (فایل stats.txt) استخراج می‌گردد. سپس، اسکریپت کنترل‌کننده با استفاده از مبدل‌های استاندارد، این داده‌های آماری را به فرمت XML مورد نیاز برای ابزار McPAT تبدیل کرده و توان مصرفی ایستا و پویا را محاسبه می‌کند. در گام بعدی، ردپای توان (Power Trace) تولید شده توسط McPAT به همراه نقشه طبقاتی (Floorplan) پردازنده، به عنوان ورودی به ابزار HotSpot داده می‌شود تا توزیع حرارتی (Temperature Map) هسته‌ها در آن بازه زمانی مشخص استخراج و در پایگاه داده ذخیره گردد.

داده‌های جمع‌آوری‌شده در بخش قبل، شامل مجموعه‌ای از سری‌های زمانی است که در آن ویژگی‌های ورودی

$$X = \{x_1, x_2, \dots, x_t\}$$

شامل رویدادشمارهای سخت‌افزاری نظیر IPC، دسترسی‌های کش و توان مصرفی یا حرارت در بازه‌های قبلی. خروجی هدف Y (توان مصرفی یا دمای هسته در بازه بعدی) نگاشت می‌شوند. این مجموعه داده برای آموزش یک مدل شبکه عصبی مبتنی بر حافظه کوتاه‌مدت ماندگار (LSTM) مورد استفاده قرار می‌گیرد. شبکه LSTM به دلیل داشتن حافظه داخلی، قادر است الگوهای زمانی و اثرات حرارتی انباشته‌شده (Thermal RC Constants) را درک کند. پس از آموزش، این مدل به عنوان یک «مدل جایگزین» سریع عمل کرده و می‌تواند بدون نیاز به اجرای مجدد McPAT و HotSpot، توان مصرفی و دمای سیستم را با دقت بالا و در کسری از ثانیه تخمین بزند.

شرح پروژه:

سیستم اطفاء حریق سالیان درازی است که به عنوان ملاکی برای پیشگیری از وقوع خسارات جانی و مالی و همچنین جلوگیری از وقوع یک فاجعه در منازل مسکونی، اداری و تجاری به وفور استفاده می شود. اما این سیستم عموماً با یک سنسور و یک سیستم اطلاع رسانی همگانی و دستگاه اطفاء طراحی می شود. اما این سیستم ایراداتی دارد و هدف ما در این پروژه طراحی یک سیستم هوشمند می باشد که این ایرادات را نداشته باشد و به خوبی وظایف یک سیستم اطفاء حریق را انجام دهد.

ایراداتی که باید برطرف شود:

- (۱) در سیستم های قدیمی عموماً از سنسور تشخیص دی اکسید کربن استفاده می شد و هر نوع غلظت این گاز را به عنوان خطر آتش در نظر می گرفت. اما گاهی اوقات این افزایش غلظت ناشی از فعالیت های شیمیایی دیگر است.
- (۲) سیستم اطفاء عموماً از نوع آب پاش های سقفی می باشد که یعنی برای هر نوع آتش از آب برای خاموش کردن آن استفاده می کند. اما گاهی اوقات آتش سوزی ناشی از منابع شیمیایی و یا قطعات الکترونیکی می باشد و آب به خوبی آن را خاموش نمی کند و حتی ممکن است آن را شعله ور تر کند.
- (۳) اطلاع رسانی همگانی سیستم اطفاء حریق به شکل آژیر های صوتی در سرتاسر ساختمان فعال می شود. اما گاهی اوقات آتش سوزی مربوط به منابع برق ساختمان می باشد که در نتیجه آن آژیر های این سیستم از کار می افتد و امکان دارد فردی از وقوع آتش سوزی آگاه نشود.

برای هر یک از ایرادات بالا یک راه حل ارائه دهید و بدون نیاز به قطعه سخت افزاری خاصی این پروژه را پیاده سازی کنید. در صورت نیاز به دوربین، میکروفون و یا اسپیکر می توانید از سخت افزار موجود در لپ تاپ و یا تلفن همراه و یا هر وسیله جانبی دیگر با این امکانات استفاده کنید.

پروژه ۳۸

دستیار آموزشی: احمد شمس

عنوان:

پیش‌بینی حرکت اتومبیل‌ها و تصمیم‌گیری جهت بارسپاری وظایف

نوع پروژه:

شبیه‌سازی

شرح پروژه: دیتاست‌های مختلفی وجود دارد که داده‌های جغرافیایی اتومبیل‌ها را در خود جای داده است. در این پروژه می‌بایست حداقل ۳ دیتاست مختلف گردآوری شود و پس از تایید توسط مسئول پروژه، پیش‌بینی حرکت اتومبیل‌ها در هر واحد زمانی صورت گیرد. هر خودرو تعدادی وظایف به صورت متناوب با زمان اجرای مشخص تولید می‌کند. همچنین در محیط تعدادی سرور به عنوان سرور لبه وجود دارد. هر خودرو وظیفه خود را دقیقاً به یک سرور برونسپاری می‌کند و نتیجه را از همان سرور دریافت می‌کند. حال یک سیستم نهفته وجود دارد که با توجه به تخمین موقعیت خودرو این سرور را انتخاب می‌کند و به خودرو اطلاع‌رسانی می‌کند. وظیفه شما شبیه‌سازی این دستگاه می‌باشد.

برای کنترل دمای اتاق سرور از یک سنسور تشخیص دما و رطوبت استفاده می‌کنیم تا فن‌ها با سرعت مشخصی به حرکت درآیند، به این صورت که مقدار دمای محیط را با استفاده از این سنسور توسط یک برد ESP32 تشخیص داده می‌شود. سپس این مقدار توسط پروتکل‌های ارتباطی Master-Slave به صورت وایرلس برای یک چیپ Rasberry Pi ارسال می‌شود. بر اساس دمای تشخیص داده شده در محیط و قاعده تعیین شده در Rasberry Pi، میزان حرکت فن‌ها را تعیین کرده و برای برد ESP32 ارسال می‌کند. برد ESP32 فن‌ها را به حرکت درآورده و دائماً فیدبک به Rasberry Pi ارسال می‌کند. در سمت Rasberry Pi اینطور فرض می‌شود که کاربری مسئول بررسی وضعیت سیستم می‌باشد. پس باید نمایشگری وجود داشته باشد تا اطلاعات کلی سیستم شامل سرعت فن‌ها و یا دمای اتاق سرور را گزارش دهد. همچنین در صورت تجاوز دما از حد مجاز باید خطاری صوتی-تصویری به کاربر بدهد که این اخطار شامل پیغام صفحه نمایش، روشن کردن چراغ هشدار قرمز و آژیر از بلندگو می‌باشد.

برای تعیین سرعت حرکت، در دمای ۱۶ درجه که دمای ایده آل محیط می‌باشد، فن‌ها خاموش در نظر گرفته می‌شوند، و به ازای هر ۲ درجه افزایش دما، ۵ درصد حداکثر سرعت موتور‌ها به سرعت حرکت آن‌ها اضافه می‌شود برای مثال در دمای ۲۴ درجه، سرعت حرکت موتور‌ها باید ۲۰ درصد سرعت کل و در دمای ۵۰ درجه ۸۵ درصد سرعت کل باید تنظیم شود. حد مجاز برای دما ۶۰ درجه سانتیگراد در نظر گرفته می‌شود.

این مقاله روشی جدید برای مدیریت تخصیص منابع و انرژی مصرفی در محاسبات ابر-لبه ارائه می‌کند. روش این مقاله که یک الگوریتم ابتکاری با پیچیدگی کم است، دستگاه‌های موبایل را به عنوان دستگاه‌های IoT در نظر گرفته و سعی در حل مصالحه‌ی بین توان مصرفی، زمان‌بندی انتقال و بارگیری دستگاه‌های موبایل دارد. در این روش تحرک دستگاه‌های موبایل نیز در نظر گرفته شده است.

پیش از شروع به پایاده‌سازی مقاله فوق می‌بایست از مقاله ارائه شده در این پروژه یک پروپوزال مناسب تهیه کنید به صورتی که ابتدا باید مقاله را به دقت مطالعه و درک عمیقی از موضوع، اهداف، روش‌ها و نتایج آن حاصل کنید. سپس با توجه به این اطلاعات، اهداف مشخص و قابل اندازه‌گیری برای پایاده‌سازی آن تعیین کنید. در این پروپوزال، علاوه بر بیان روش مقاله و محدودیت‌های موجود، زبان برنامه‌نویسی و نرم‌افزارهای مناسب برای پایاده‌سازی نیز معرفی و دلایل انتخاب آن را ذکر کنید. همچنین، خلاصه‌ای از مقاله ارائه شده و نحوه پایاده‌سازی آن گام‌به‌گام شرح داده شود.

این مقاله روشی جدید برای بهینه‌سازی بارسپاری و تخصیص منابع در محاسبات لبه موبایل ارائه می‌کند. هدف این روش، حل مصالحه‌ی بین بارسپاری، تخصیص منابع و انرژی مصرفی در محیط‌های پویای محاسبات لبه موبایل می‌باشد. این روش با استفاده از یک مدل یادگیری تقویتی عمیق سعی در رسیدن به اهداف فوق و حل این مصالحه دارد.

پیش از شروع به پایاده‌سازی مقاله فوق می‌بایست از مقاله ارائه شده در این پروژه یک پروپوزال مناسب تهیه کنید به صورتی که ابتدا باید مقاله را به‌دقت مطالعه و درک عمیقی از موضوع، اهداف، روش‌ها و نتایج آن حاصل کنید. سپس با توجه به این اطلاعات، اهداف مشخص و قابل اندازه‌گیری برای پایاده‌سازی آن تعیین کنید. در این پروپوزال، علاوه بر بیان روش مقاله و محدودیت‌های موجود، زبان برنامه‌نویسی و نرم‌افزارهای مناسب برای پایاده‌سازی نیز معرفی و دلایل انتخاب آن را ذکر کنید. همچنین، خلاصه‌ای از مقاله ارائه شده و نحوه پایاده‌سازی آن گام‌به‌گام شرح داده شود.

این مقاله یک چارچوب مدیریت منابع حرارتی به نام RT-TRM را پیشنهاد می دهد که برای سیستم های نهفته بی درنگ چندهسته ای، به ویژه در کاربردهای خودرویی، طراحی شده است. این چارچوب با استفاده از مدل توان پویای وظایف و برنامه ریزی آنلاین زمان های بیکاری، دمای پردازنده را در محدوده ای ایمن نگه داشته و در عین حال، محدودیت های زمان بندی را رعایت می کند.

پیش از شروع به پایاده سازی مقاله فوق می بایست از مقاله ارائه شده در این پروژه یک پروپوزال مناسب تهیه کنید به صورتی که ابتدا باید مقاله را به دقت مطالعه و درک عمیقی از موضوع، اهداف، روش ها و نتایج آن حاصل کنید. سپس با توجه به این اطلاعات، اهداف مشخص و قابل اندازه گیری برای پایاده سازی آن تعیین کنید. در این پروپوزال، علاوه بر بیان روش مقاله و محدودیت های موجود، زبان برنامه نویسی و نرم افزارهای مناسب برای پایاده سازی نیز معرفی و دلایل انتخاب آن را ذکر کنید. همچنین، خلاصه ای از مقاله ارائه شده و نحوه پایاده سازی آن گام به گام شرح داده شود.

در این مقاله، یک سامانه پردازش لبه سیار شامل دستگاه‌های کاربر، سرورهای لبه و سرور ابری مدل می‌شود و هر وظیفه به‌صورت یک گراف DAG شامل زیروظایف وابسته نمایش داده می‌شود. هدف مقاله این است که با استفاده از روش FEDG، ویژگی‌های وابستگی بین زیروظایف استخراج شود و سپس با الگوریتم DGMA مبتنی بر یادگیری تقویتی عمیق چندعامله، برای هر زیروظیفه تصمیم گرفته شود که به‌صورت محلی، روی سرور لبه یا روی ابر اجرا شود و چه مقدار منبع پردازشی به آن اختصاص یابد. در این پروژه باید بخش‌های اصلی مقاله شامل مدل شبکه MEC، مدل DAG، مدل ارتباطی، مدل تأخیر، مدل مصرف انرژی، تابع utility، استخراج ویژگی با GNN، الگوریتم DGMA، بازپخش تجربه اولویت‌دار، اشتراک پارامتر انتخابی بین عامل‌ها و baseline‌های مقاله پایاده‌سازی شوند. خروجی نهایی پروژه شامل کد کامل شبیه‌سازی، بازتولید نمودارهای اصلی مقاله مانند تأخیر، مصرف انرژی، utility، نرخ نقض deadline و روند همگرایی، و تهیه گزارش کامل از روش پایاده‌سازی و تحلیل نتایج خواهد بود.

پیش از شروع به پایاده‌سازی مقاله فوق می‌بایست از مقاله ارائه شده در این پروژه یک پروپوزال مناسب تهیه کنید به صورتی که ابتدا باید مقاله را به‌دقت مطالعه و درک عمیقی از موضوع، اهداف، روش‌ها و نتایج آن حاصل کنید. سپس با توجه به این اطلاعات، اهداف مشخص و قابل اندازه‌گیری برای پایاده‌سازی آن تعیین کنید. در این پروپوزال، علاوه بر بیان روش مقاله و محدودیت‌های موجود، زبان برنامه‌نویسی و نرم‌افزارهای مناسب برای پایاده‌سازی نیز معرفی و دلایل انتخاب آن را ذکر کنید. همچنین، خلاصه‌ای از مقاله ارائه شده و نحوه پایاده‌سازی آن گام‌به‌گام شرح داده شود.

این مقاله مشکل اصلی ROS 2 در کاربردهای بلادرنگ را به پیاده‌سازی executor نسبت می‌دهد؛ زیرا executor پیش‌فرض ROS 2 از preemption و اولویت‌دهی کاربرمحور به callback پشتیبانی کافی ندارد. هدف مقاله ارائه چارچوب executor جدیدی به نام ROS {RT} است که با انواع برنامه‌های ROS 2 سازگار بوده و زمان‌بندی preemptive و priority-driven را امکان‌پذیر می‌کند. همچنین مقاله برای پشتیبانی از زمان‌بندی EDF، یک scheduler سفارشی مبتنی بر قابلیت جدید Linux با نام SCHED_EXT ارائه می‌دهد. در این پروژه باید executor پیشنهادی مقاله در ROS 2 پیاده‌سازی شود، امکان تعیین اولویت و deadline برای callbackها اضافه گردد، مکانیزم preemption و زمان‌بندی priority-based و EDF بازتولید شود، و کارایی آن با executor پیش‌فرض ROS 2 مقایسه گردد. خروجی نهایی پروژه شامل پیاده‌سازی ROS {RT}، اجرای چند workload آزمایشی شامل publisher/subscriber و در صورت امکان سناریوی LiDAR مشابه Autoware، رسم نمودارهای تأخیر، overhead، response time و deadline miss ratio، و تهیه گزارش کامل از معماری، پیاده‌سازی، آزمایش‌ها و تحلیل نتایج خواهد بود.

پیش از شروع به پیاده‌سازی مقاله فوق می‌بایست از مقاله ارائه شده در این پروژه یک پروپوزال مناسب تهیه کنید به صورتی که ابتدا باید مقاله را به‌دقت مطالعه و درک عمیقی از موضوع، اهداف، روش‌ها و نتایج آن حاصل کنید. سپس باتوجه به این اطلاعات، اهداف مشخص و قابل‌اندازه‌گیری برای پیاده‌سازی آن تعیین کنید. در این پروپوزال، علاوه بر بیان روش مقاله و محدودیت‌های موجود، زبان برنامه‌نویسی و نرم‌افزارهای مناسب برای پیاده‌سازی نیز معرفی و دلایل انتخاب آن را ذکر کنید. همچنین، خلاصه‌ای از مقاله ارائه شده و نحوه پیاده‌سازی آن گام‌به‌گام شرح داده شود.